BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC FPT

ĐÁNH GIÁ ẢNH HƯỞNG CỦA TIỀN XỬ LÝ ẢNH ĐẾN PHÁT HIỆN PHƯƠNG TIỆN TRÊN BDD100K BẰNG YOLO26n

BÁO CÁO ĐỒ ÁN HỌC PHẦN

Image and Video Processing — Xử lý ảnh và video

Nhóm: [Tên nhóm / lớp học phần]

[Họ tên thành viên 1] — [MSSV]

[Họ tên thành viên 2] — [MSSV]

[Họ tên thành viên 3] — [MSSV]

[Họ tên thành viên 4] — [MSSV]

[Họ tên thành viên 5] — [MSSV]

[Họ tên thành viên 6] — [MSSV]

Việt Nam, tháng 11 năm 2026

© 2026 [Tên nhóm / tác giả]

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC FPT

ĐÁNH GIÁ ẢNH HƯỞNG CỦA TIỀN XỬ LÝ ẢNH ĐẾN PHÁT HIỆN PHƯƠNG TIỆN TRÊN BDD100K BẰNG YOLO26n

BÁO CÁO ĐỒ ÁN HỌC PHẦN

Image and Video Processing — Xử lý ảnh và video

Nhóm: [Tên nhóm / lớp học phần]

[Họ tên thành viên 1] — [MSSV]

[Họ tên thành viên 2] — [MSSV]

[Họ tên thành viên 3] — [MSSV]

[Họ tên thành viên 4] — [MSSV]

[Họ tên thành viên 5] — [MSSV]

[Họ tên thành viên 6] — [MSSV]

Giảng viên hướng dẫn: Dr. Nguyen Ngoc Truong Minh

Báo cáo nhóm đáp ứng yêu cầu của học phần Image and Video Processing

Việt Nam, tháng 11 năm 2026

© 2026 [Tên nhóm / tác giả]

TÓM TẮT

Báo cáo phân tích năm chính sách tiền xử lý ảnh cho phát hiện phương tiện BDD100K bằng YOLO26n, biến thể nano khởi tạo từ COCO: Raw, Gamma, Local-shadow, CLAHE thích nghi và Canny. Mô hình học trên 9.781 ảnh, chọn checkpoint bằng calibration và đánh giá trên 13.975 ảnh development. Nhóm notebook batch32 có đủ v0–v3: v2 đạt mAP@50 41,9615%; v3 đạt 42,1348%, hơn Raw 0,0999 điểm phần trăm, nhưng mAP@50:95 giảm nhẹ. Ở nhóm batch16, Canny đạt mAP@50 42,0345%, thấp hơn Raw 42,5961%. Báo cáo phân tích từng lớp, cohort, định tuyến, hội tụ và nguồn chứng cứ; phân biệt các nhóm cấu hình trước khi kết luận. Kết quả một seed chưa xác nhận ưu thế ổn định; final holdout tiếp tục được giữ kín.

Từ khóa: BDD100K; YOLO26n; tiền xử lý ảnh; Gamma; CLAHE; Canny; phát hiện phương tiện.

Phạm vi báo cáo: phương pháp v0–v4 và các lượt chạy đồng bộ đã có chứng cứ. Số liệu được chốt tại 2026-11-12 15:39:25 (UTC+7). Những bước đang chạy được ghi rõ trạng thái; không sử dụng điểm calibration thay cho kết quả development.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 1

1.1. Bối cảnh và bài toán 1

1.2. Mục tiêu nghiên cứu 1

1.3. Phạm vi và đóng góp 1

1.4. Tổ chức báo cáo 2

CHƯƠNG 2. TỔNG QUAN NGHIÊN CỨU 3

2.1. Mô hình YOLO26n sử dụng trong nghiên cứu 3

2.2. Gamma và điều chỉnh chiếu sáng 9

2.3. CLAHE và tương phản cục bộ 9

2.4. Canny và giả thuyết tăng chi tiết theo biên 9

2.5. Khoảng trống cần kiểm chứng 10

CHƯƠNG 3. PHƯƠNG PHÁP NGHIÊN CỨU 11

3.1. BDD100K và dữ liệu sử dụng 11

3.2. Nhãn năm lớp và toàn vẹn hình học 14

3.3. Đặc trưng chất lượng và quy ước chung 15

3.4. Các chính sách v0–v4 17

3.5. Cấu hình huấn luyện và chọn checkpoint 23

3.6. Thước đo đánh giá 24

3.7. Quy trình notebook và toàn vẹn chứng cứ 27

3.8. Hai nhóm full fine-tune và nguồn chứng cứ 29
3.9. Cơ chế, giả thuyết và điểm nhạy của từng toán tử 30
3.10. Pseudocode và hợp đồng dữ liệu 32
3.11. Chia nhóm và giới hạn bảo vệ rò rỉ 32
3.12. Từ ảnh nguồn tới tensor của YOLO26n 33
3.13. Mục tiêu học, cập nhật trọng số và chọn model 34
3.14. Đại lượng cần ước lượng và ma trận đối chứng 36
3.15. Phân rã ảnh hưởng train và ảnh hưởng input: thiết kế mở rộng 37
3.16. Kiểm chứng tính nhất quán và kế hoạch độ bất định 37
3.17. Đo chi phí và giới hạn thực nghiệm 38
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 39
4.1. Bản đồ chứng cứ và nguyên tắc đọc kết quả 39
4.2. Nhóm A: kết quả tổng thể v0–v3 39
4.3. Nhóm A: AP theo lớp và mất cân bằng hỗ trợ 41
4.4. Nhóm A: cohort tối/sáng và định tuyến 44
4.5. Nhóm A: lựa chọn checkpoint, hội tụ và thời gian 46
4.6. Nhóm B: đối chiếu v4 với Raw/Gamma cùng recipe 48
4.7. Đối chiếu giữa nhóm A và B: độ ổn định của kết luận 52
4.8. Kết quả lịch sử trong issue và phạm vi suy luận 52
4.9. Trả lời câu hỏi nghiên cứu và giới hạn 53

4.10. Diễn giải kết quả theo cơ chế của detector 54

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 55

5.1. Kết luận từ năm phiên bản 55

5.2. Lộ trình kiểm chứng tiếp theo 55

TÀI LIỆU THAM KHẢO 57

PHỤ LỤC 58

A. Bảng đầy đủ Precision, Recall, F1 và AP theo lớp 58

B. Truy vết notebook và tọa độ output 59

C. Truy vết kết quả mới và dữ liệu cố định 61

D. Cấu hình cần đối chiếu khi tái lập 62

E. Nguồn lịch sử và cách diễn giải 63

F. Hạn chế và hoàn thiện hồ sơ nộp 63

G. Hồ sơ model và cách đọc mã train 63

H. Nguồn chi tiết cho Chương3 64

DANH MỤC BẢNG

Bảng 1. Các thuật ngữ và đơn vị sử dụng xi

Bảng 2. Định danh chính xác detector dùng để train 3

Bảng 3. Các khối trong kiến trúc cục bộ và vai trò 4

Bảng 4. Ba thang đặc trưng của YOLO26n — ví dụ tensor 640×640 5

Bảng 5. Ánh xạ ngữ nghĩa COCO sang taxonomy đầu ra của đồ án 7

Bảng 6. Số tham số đọc trực tiếp từ checkpoint trên CPU 8

Bảng 7. Quần thể và nhóm của bảng chia cố định 12

Bảng 8. Quyền sử dụng từng quần thể trong nghiên cứu 13

Bảng 9. Ánh xạ nhãn detection sang bài toán năm lớp 14

Bảng 10. Đặc trưng, đơn vị và vai trò trong gate 16

Bảng 11. Các tình huống biên minh họa theo toán tử so sánh trong mã 17

Bảng 12. Tổng hợp chính sách đã triển khai 17

Bảng 13. Giả mã v0 và v1 20

Bảng 14. Giả mã v2 — bản đồ gain theo vị trí 21

Bảng 15. Giả mã v3 — offset CLAHE có bảo vệ 21

Bảng 16. Giả mã v4 — Canny-guided detail 22

Bảng 17. Recipe native chung của các lượt so sánh đồng bộ 23

Bảng 18. Ví dụ ghép tại IoU 0,50: một lớp, hai GT trong cùng ảnh 25

Bảng 19. Các ngưỡng không thể thay thế cho nhau 26

Bảng 20. Trạng thái thực nghiệm và chứng cứ cần có 27

Bảng 21. Các trường xuất trong metadata.csv 28

Bảng 22. Phân biệt nhóm thực nghiệm và mức độ kiểm chứng 29

Bảng 23. Giả thuyết cải thiện và kiểm tra có thể bác bỏ 30

Bảng 24. Các mốc của hàm gain v2 trước nội suy 30

Bảng 25. Luồng chung có thể tái lập từ notebook 01–04 32

Bảng 26. Loss và ý nghĩa khi đọc đường cong 34

Bảng 27. Diễn giải chu kỳ train của từng policy 35

Bảng 28. Các cặp so sánh được phép trong snapshot đang báo cáo 36

Bảng 29. Ma trận factorial 2×2 được đề xuất cho mỗi policy 37

Bảng 30. Các kiểm tra để chấp nhận một bảng kết quả 37

Bảng 31. Kiểm kê kết quả v0–v4 trong project 39

Bảng 32. Nhóm A — development 13.975 ảnh, 152.774 GT; đơn vị % 39

Bảng 33. Nhóm A — chênh lệch từng policy so với A-v0, đơn vị pp 40

Bảng 34. Hỗ trợ GT development của năm lớp 41

Bảng 35. Nhóm A — AP@50 theo lớp, đơn vị % 42

Bảng 36. Nhóm A — AP@50:95 theo lớp, đơn vị % 42

Bảng 37. A-v2 so với A-v0 — từng lớp, đơn vị pp 43

Bảng 38. A-v3 so với A-v0 — từng lớp, đơn vị pp 44

Bảng 39. Nhóm A — cohort cùng định nghĩa theo ảnh gốc, mAP@50:95 (%) 44

Bảng 40. Nhóm A — tỷ lệ Apply theo tập, đơn vị % 45

Bảng 41. Nhóm A — cohort định tuyến đã lưu, mAP@50:95 (%) 46

Bảng 42. Nhóm A — tóm tắt huấn luyện từ output 03_train 46

Bảng 43. Nhóm A — tốc độ native evaluator, ms/ảnh 48

Bảng 44. Nhóm B — kết quả development đã hoàn tất, đơn vị % 48

Bảng 45. Nhóm B — AP@50 từng lớp (%) và delta v4−Raw 49

Bảng 46. Nhóm B — AP@50:95 từng lớp (%) và delta v4−Raw 49

Bảng 47. Nhóm B — cohort cố định, mAP@50:95 (%) 50

Bảng 48. Nhóm B — train và native inference của các lượt hoàn tất 51

Bảng 49. Cùng policy, khác nhóm thực thi; đơn vị % 52

Bảng 50. Lịch sử Issue45 — S20/640, 20 epoch, freeze11, AdamW FP32 52

Bảng 51. Tổng hợp trả lời RQ từ chứng cứ hiện có 53

Bảng 52. Từ quan sát đến giả thuyết: tránh suy luận vượt dữ liệu 54

Bảng 53. Các bước tiếp theo và tiêu chí hoàn tất 55

Bảng 54. Nhóm A — v0: metric native đầy đủ theo lớp (%) 58

Bảng 55. Nhóm A — v1: metric native đầy đủ theo lớp (%) 58

Bảng 56. Nhóm A — v2: metric native đầy đủ theo lớp (%) 58

Bảng 57. Nhóm A — v3: metric native đầy đủ theo lớp (%) 59

Bảng 58. Chỉ dẫn đọc kết quả trong từng notebook v0–v3 59

Bảng 59. Checklist cấu hình và lý do phải giữ cố định 62

Bảng 60. Các file cần đọc để hiểu detector và thực nghiệm 64

Bảng 61. Ánh xạ nội dung methodology sang hàm thực thi 65

DANH MỤC HÌNH

Hình 1. Sơ đồ YOLO26n từ YAML và checkpoint cục bộ. Ba kích thước feature map minh họa cho tensor vuông 640×640, không khẳng định mọi batch eval có hình dạng này. 5

Hình 2. Luồng transfer learning thực tế: cùng checkpoint COCO khởi tạo độc lập cho từng policy; calibration chọn checkpoint, development báo cáo metric. 7

Hình 3. Thiết kế phương pháp: policy tác động lên pixel trước dataloader; calibration chọn model; development đo kết quả. Final holdout được tách khỏi luồng so sánh. 11

Hình 4. Quy trình dữ liệu và vai trò của từng tập; final holdout không tham gia chọn chính sách, checkpoint hoặc báo cáo AP. 12

Hình 5. Ví dụ calibration 05391f8a-a0e258ec với 12 hộp GT năm lớp. Đây là nhãn tham chiếu, không phải dự đoán của mô hình. 15

Hình 6. Ảnh calibration 6bbb63a9-5d1b6029 qua v0–v4, chọn từ mẫu seed2026 theo độ sáng, không theo kết quả detector. Hình là preview pixel trước JPEG95, không phải bằng chứng tăng AP. 19

Hình 7. Chi tiết v4 trên cùng ảnh calibration: bản đồ Canny chỉ dùng để hướng dẫn mặt nạ; detector nhận ảnh màu. Thang thay đổi 0–12 áp dụng trước JPEG. 20

Hình 8. Luồng v4 theo apply_recipe: Canny được tính để đo gate cả khi ảnh cuối cùng Skip; biên được tái sử dụng khi Apply. 22

Hình 9. Quy trình đánh giá native: matching từng ảnh và từng IoU, xếp confidence theo lớp, tính AP rồi tổng hợp. Cùng thống kê phục vụ phân tích cohort. 27

Hình 10. Ví dụ hình học khi letterbox vuông: ảnh1280×720 thu về640×360 rồi pad140pixel mỗi phía trên/dưới. Đây là minh họa; rect=True có thể dùng canvas chữ nhật thay vì640×640. 33

Hình 11. Nhóm A: tổng hợp số liệu từ output notebook 04_eval, cùng batch32, 50 epoch và seed2026. 40

Hình 12. Nhóm A: delta được tính bằng 100×(metric_policy−metric_Raw); đường 0 là đối chứng. 41

Hình 13. Phân bố GT development trên thang log; một ảnh có thể chứa nhiều lớp nên số ảnh theo lớp không cộng thành tổng ảnh. 42

Hình 14. Nhóm A: AP@50 và AP@50:95 của từng lớp từ các bảng output đã lưu; hai biểu đồ dùng thang màu khác nhau. 43

Hình 15. Nhóm A: dark/bright là ngưỡng brightness, không đồng nghĩa trực tiếp với metadata night/day. 45

Hình 16. Nhóm A — v0: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có. 47

Hình 17. Nhóm A — v1: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có. 47

Hình 18. Nhóm A — v2: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có. 47

Hình 19. Nhóm A — v3: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có. 48

Hình 20. Nhóm B: chỉ những lượt đã hoàn tất có điểm; các lượt v2/v3 của nhóm A đã được báo cáo riêng phía trên. 49

Hình 21. Nhóm B: AP@50 từng lớp; số GT development giống nhóm A. 50

Hình 22. Nhóm B: đối chiếu Raw/v4 trên cùng cohort theo brightness ảnh gốc. 51

Hình 23. Nhóm B: đường cong Raw/v4 lấy từ results.csv native, không trộn với đường cong notebook nhóm A. 51

DANH MỤC CHỮ VIẾT TẮT

Bảng 1. Các thuật ngữ và đơn vị sử dụng

Ký hiệu	**Diễn giải**
AP / mAP	Average Precision / trung bình AP trên các lớp
IoU	Intersection over Union — tỷ lệ giao trên hợp hai hộp
GT	Ground truth — nhãn tham chiếu
CLAHE	Cân bằng histogram thích nghi có giới hạn tương phản
AMP	Huấn luyện hỗn hợp độ chính xác
NMS	Non-Maximum Suppression
S10 / S20	Hai tập con huấn luyện lồng nhau đã cố định của dự án
pp	Điểm phần trăm; khác với phần trăm thay đổi tương đối
Apply / Skip	Áp dụng toán tử / giữ nguyên pixel trước mã hóa JPEG95

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và bài toán

Phát hiện phương tiện từ camera hành trình là một thành phần của các hệ thống phân tích giao thông. Ảnh thực tế có thể chứa vùng tối, bóng đổ, nền nhiều chi tiết, phương tiện nhỏ và các trường hợp che khuất. Một thao tác tăng sáng hoặc tăng tương phản giúp con người nhìn rõ ảnh hơn chưa chắc làm mô hình nhận dạng tốt hơn. Do đó, việc đánh giá preprocessing cần dựa trên độ chính xác phát hiện cùng chi phí tính toán, thay vì chỉ quan sát ảnh sau xử lý.

BDD100K cung cấp dữ liệu cảnh lái xe đa dạng về thời gian, thời tiết và môi trường; công trình gốc mô tả 100.000 video và nhiều bài toán thị giác liên quan [1]. Trong đồ án này, dữ liệu được khai thác cho phát hiện hộp giới hạn của phương tiện, không mở rộng sang phân đoạn, theo dõi hay ước lượng quỹ đạo.

1.2. Mục tiêu nghiên cứu

Mục tiêu là xây dựng quy trình so sánh có kiểm soát giữa ảnh Raw và bốn chính sách tiền xử lý ngoại tuyến: Gamma-or-Skip, Local-shadow, tăng tương phản cục bộ thích nghi và tăng chi tiết có hướng dẫn từ Canny. Mỗi chính sách đi kèm một YOLO26n được fine-tune độc lập từ cùng checkpoint khởi tạo trên chính phân bố ảnh đó.

Ba câu hỏi nghiên cứu được đặt ra: (RQ1) chính sách có cải thiện mAP@50 và mAP@50:95 so với Raw ở cùng cấu hình hay không; (RQ2) thay đổi tổng thể có che lấp suy giảm ở motorcycle và bicycle hay không; (RQ3) cơ chế định tuyến Apply/Skip và chi phí bổ sung có phù hợp với lợi ích quan sát được hay không. Báo cáo không giả định trước rằng một phiên bản mới hơn sẽ tốt hơn.

1.3. Phạm vi và đóng góp

Phạm vi chính là năm phiên bản v0–v4 của bản làm việc cục bộ. Dữ liệu, seed, checkpoint, phân chia vai trò và nhãn được cố định; chỉ chính sách pixel là yếu tố nghiên cứu. Những điều chỉnh batch/worker do giới hạn phần cứng được công bố và được đồng bộ ở các lượt so sánh. Những kết

quả lịch sử dùng frozen backbone hoặc độ phân giải khác không được trộn vào bảng full fine-tune hiện tại.

Đóng góp gồm mô tả tái lập từng toán tử và điều kiện định tuyến; thiết kế dữ liệu chống rò rỉ theo nhóm nguồn; quy trình notebook độc lập từ preview đến đánh giá; và phân tích kết quả thực tế, kể cả kết quả âm. Những phiên bản chưa có metric development được giữ trạng thái chưa hoàn tất, thay vì suy đoán điểm số từ cấu hình hoặc đường cong calibration.

1.4. Tổ chức báo cáo

Chương 2 giới thiệu YOLO26n, backbone/neck/head, dự đoán nhiều thang và transfer learning, sau đó tổng hợp cơ sở các toán tử xử lý ảnh. Chương 3 trình bày BDD100K, dữ liệu thực nghiệm, thuật toán v0–v4 và cấu hình huấn luyện/đánh giá. Chương 4 báo cáo kết quả đã có chứng cứ và giới hạn diễn giải. Chương 5 kết luận và đề xuất bước kiểm chứng tiếp theo. Phụ lục cung cấp đường dẫn chứng cứ, dấu vết hash và hướng dẫn tái lập.

CHƯƠNG 2. TỔNG QUAN NGHIÊN CỨU

2.1. Mô hình YOLO26n sử dụng trong nghiên cứu

2.1.1. Giới thiệu và nhiệm vụ của mô hình

YOLO là viết tắt của You Only Look Once, một họ mô hình phát hiện đối tượng. Thay vì chỉ gán một nhãn cho cả ảnh, detector xác định nhiều đối tượng trong cùng khung hình và trả về vị trí của từng đối tượng. Nghiên cứu sử dụng Ultralytics YOLO26n, biến thể nano của dòng YOLO26, cho nhiệm vụ object detection [3], [11].

Trong đồ án này, một ảnh đường phố có thể chứa nhiều car, truck, bus, motorcycle và bicycle. Đầu ra mong muốn là một danh sách hộp bao: mỗi hộp có tọa độ, nhãn phương tiện và điểm tin cậy. Bài toán không yêu cầu mặt nạ pixel hay mã định danh xuyên suốt video; do đó mô hình được train theo tác vụ detect. Quy trình hiện tại đánh giá từng ảnh, chưa cung cấp thành phần tracking hay ước lượng khoảng cách.

Bảng 2. Định danh chính xác detector dùng để train

Thuộc tính	**Giá trị trong project**
Họ / biến thể	Ultralytics YOLO26 / YOLO26n (nano)
Tác vụ	Object detection: hộp bao và lớp phương tiện
Thư viện	ultralytics==8.4.198 trong các nhóm được báo cáo
Khởi tạo	weights/yolo26n.pt, pretrained COCO-80; SHA256 được cố định
Đầu ra sau fine-tune	5 lớp: car, truck, bus, motorcycle, bicycle
Recipe chính	Full fine-tune 50 epoch; freeze=0; optimizer=SGD
Độ phân giải danh nghĩa	imgsz=640; train rect=False, eval rect=True
Chế độ đánh giá	nms=False; FP32; max_det=300; development

2.1.2. Ý nghĩa của biến thể nano và phạm vi lựa chọn

Hậu tố n chỉ biến thể nano. YAML cục bộ còn định nghĩa các mức s, m, l và x với hệ số độ sâu/độ rộng khác nhau; n dùng depth=0,50 và width=0,25. Việc giữ cùng biến thể n cho mọi policy giúp thí nghiệm tập trung vào tiền xử lý. Nếu đổi từ n sang s rồi so AP, phần chênh lệch có thể đến từ dung lượng mạng chứ không chỉ từ ảnh đầu vào [13].

Trong điều kiện một GPU GTX1660Ti 6 GiB ở nhóm B, chọn một mạng quy mô nhỏ giúp tổ chức nhiều lần train độc lập với ngân sách hiện có. Đây là lý do thực dụng của thiết kế; nghiên cứu chưa thực hiện benchmark n/s/m/l/x để chứng minh n là lựa chọn tốt nhất. Từ nano cũng không bảo đảm đủ tốc độ cho mọi hệ thống thực tế, vì độ trễ còn phụ thuộc phần cứng, backend và toàn bộ luồng xử lý.

Cần phân biệt phiên bản phần mềm, phiên bản kiến trúc và phiên bản phương pháp: 8.4.198 là phiên bản package Ultralytics; YOLO26n là model; v0–v4 là năm policy preprocessing của đồ án. Mọi policy trong báo cáo giữ cùng họ detector. Tên v4 không có nghĩa sử dụng YOLOv4; tương tự v0 không phải một kiến trúc YOLO riêng.

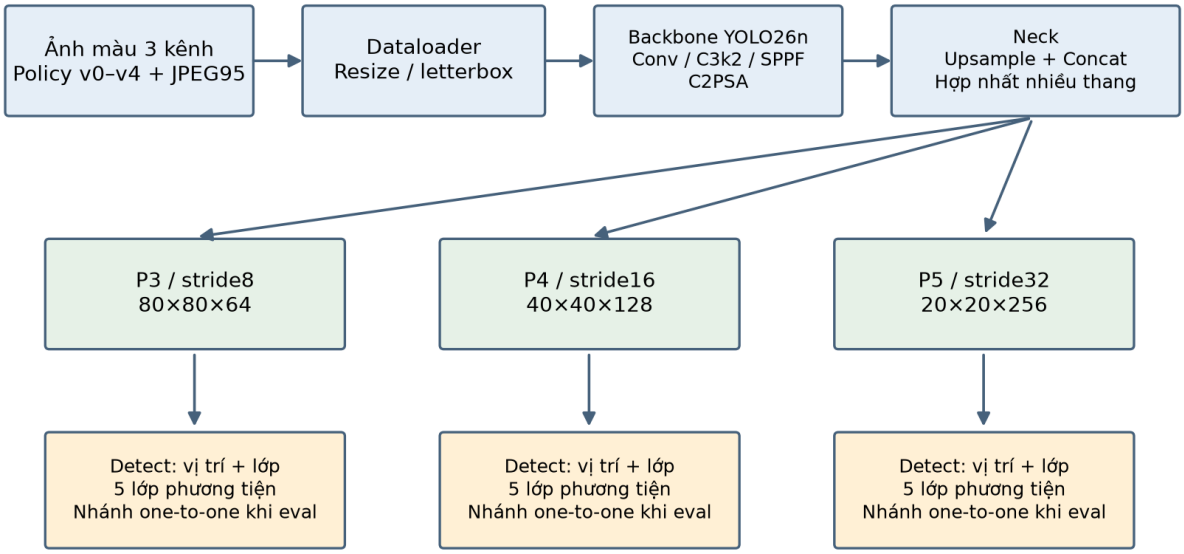
2.1.3. Cấu trúc tổng quan: backbone, neck và detection head

Dựa trên yolo26.yaml và checkpoint đọc trực tiếp trên máy, có thể mô tả mạng thành ba khối chức năng. Backbone biến ảnh thành các bản đồ đặc trưng; neck kết hợp thông tin giữa nhiều độ phân giải; detection head suy ra vị trí và điểm lớp. Cách chia này giúp giải thích luồng tính toán, dù YAML của Ultralytics gộp phần neck vào danh sách head thay vì dùng một khóa neck riêng [13].

Bảng 3. Các khối trong kiến trúc cục bộ và vai trò

Khối	Module / thao tác quan sát được	Vai trò trong detector
Backbone	Conv, C3k2, SPPF, C2PSA; lớp0–10	Rút trích đặc trưng và tích hợp ngữ cảnh từ ảnh
Neck	Upsample, Concat, C3k2, Conv; lớp11–22	Kết hợp đặc trưng có độ phân giải và mức trừu tượng khác nhau
Detect	Lớp23; nhận các đầu ra16/19/22	Dự đoán hộp và điểm lớp ở ba thang
Nhánh dự đoán	cv2/cv3 và one2one_cv2/one2one_cv3	Tách hồi quy hộp / phân lớp; có hai cơ chế gán nhãn khi học

Conv thực hiện biến đổi tích chập; các bước stride2 giảm độ phân giải không gian. Các khối C3k2 tạo chuỗi xử lý đặc trưng có kết nối nội bộ; SPPF dùng pooling để gom ngữ cảnh; C2PSA tích hợp xử lý attention trong cấu trúc được khai báo. Báo cáo không coi riêng một khối là nguyên nhân làm AP tăng: để xác định đóng góp từng khối cần ablation kiến trúc, trong khi nghiên cứu này giữ model cố định.



Hình 1. Sơ đồ YOLO26n từ YAML và checkpoint cục bộ. Ba kích thước feature map minh họa cho tensor vuông 640×640, không khẳng định mọi batch eval có hình dạng này.

2.1.4. Dự đoán nhiều thang và phương tiện có kích thước khác nhau

Detection head nhận đặc trưng P3/P4/P5 với stride8/16/32. Stride biểu thị mức giảm kích thước so với tensor đầu vào. Với ảnh mạng 640×640, các kích thước không gian tương ứng là 80×80, 40×40 và 20×20. Checkpoint nano hiện có số kênh vào head lần lượt 64, 128 và 256 [13].

Bảng 4. Ba thang đặc trưng của YOLO26n — ví dụ tensor 640×640

Thang	Stride	H×W	Kênh vào Detect	Ý nghĩa
P3	8	80×80	64	Lưới dày hơn, giữ thông tin không gian chi tiết hơn
P4	16	40×40	128	Kết hợp độ phân giải và ngữ cảnh
P5	32	20×20	256	Đặc trưng thô hơn với ngữ cảnh rộng hơn

Ví dụ hình học: một phương tiện rộng16 pixel sau resize chỉ trải khoảng hai bước stride8; nếu vật thể còn nhỏ hơn, thông tin để mô hình định vị rất hạn chế. Đây là ví dụ giải thích, không phải thống kê kích thước GT của dataset. Làm nét hoặc Canny có thể thay tương phản nhưng không tạo lại chi tiết vật thể đã mất khi giảm kích thước. Vì vậy cần đánh giá preprocessing cùng độ phân giải detector, không chỉ nhìn ảnh nguồn.

Ba thang không có nghĩa lớp car bị gán cố định vào một thang và motorcycle vào thang khác. Một loại phương tiện có thể xuất hiện ở nhiều kích thước. Tổng số vị trí trên ba lưới vuông là $80^2+40^2+20^2=8.400$; đó là số vị trí dự đoán trước giải mã/chọn lọc, không phải 8.400 xe trong ảnh hay số hộp đầu ra cuối. Khi eval dùng tensor chữ nhật, số vị trí sẽ thay đổi theo kích thước thực tế.

2.1.5. Hộp bao, điểm lớp và chế độ one-to-one

Với mỗi ứng viên, head tạo thông tin hồi quy hộp và điểm cho các lớp. Sau giải mã, hộp có thể biểu diễn bằng (x1,y1,x2,y2), kèm confidence và class_id. Điểm confidence phản ánh mức tự tin của dự đoán theo mô hình; nó không tự động là xác suất đã hiệu chuẩn. Một hộp confidence cao vẫn có thể sai lớp, sai vị trí hoặc trùng đối tượng khác.

Checkpoint cục bộ có cả nhánh one-to-many và one-to-one. Chế độ đầu cho phép nhiều vị trí học quanh một đối tượng; chế độ sau phục vụ chọn dự đoán gọn hơn. Trong BaseValidator của package8.4.198, args.nms=False yêu cầu backend dùng nhánh end-to-end. Vì thế báo cáo ghi rõ nms=False khi mô tả đánh giá, thay vì suy từ tên YOLO rằng mọi cách chạy đều dùng cùng cơ chế hậu xử lý [13].

Ngay sau khi chỉ nạp checkpoint bằng YOLO(path), thuộc tính end2end có thể đang False do chế độ mặc định của đối tượng Python, dù model có nhánh one-to-one. Đây không phải bằng chứng lần đánh giá trong project đã dùng nhánh khác: validator sẽ thiết lập chế độ từ args.nms. model_evidence.json ghi riêng trạng thái ngay sau load và sự hiện diện các nhánh, tránh nhập nhằng giữa metadata tĩnh và chế độ runtime.

NMS truyền thống loại các hộp có độ chồng lặp cao theo quy tắc hậu xử lý. Với luồng end-to-end được yêu cầu ở đây, không áp dụng bước NMS loại hộp chồng như luồng one-to-many; vẫn có giải mã, chọn lọc confidence và giới hạn số hộp. Tham số iou=0,7 được giữ trong cấu hình thư viện, còn các ngưỡng IoU0,50…0,95 dùng tính AP là một khái niệm khác. Không gọi iou=0,7 là ngưỡng mAP@50.

2.1.6. Chuyển từ COCO-80 sang năm lớp BDD100K

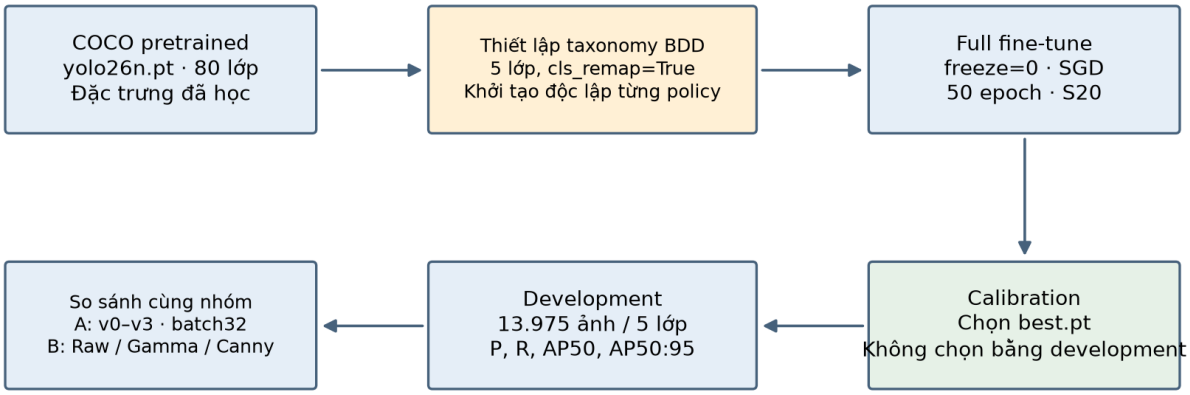
Pretrained nghĩa là trọng số khởi tạo đã được học trước trên một nhiệm vụ nguồn. Dự án bắt đầu từ yolo26n.pt có taxonomy COCO80, sau đó dùng dữ liệu BDD năm lớp để fine-tune. Mục đích

là sử dụng đặc trưng đã học làm điểm xuất phát, đồng thời thích nghi detector với bài toán phương tiện. Đây là transfer learning; khác với train from scratch từ trọng số ngẫu nhiên [12], [13].

Bảng 5. Ánh xạ ngữ nghĩa COCO sang taxonomy đầu ra của đồ án

Lớp	COCO ID (0-based)	BDD output ID	Xác minh trong mã
car	2	0	verify_coco_names kiểm tra tên nguồn
truck	7	1	Giữ đúng ngữ nghĩa khi đổi thứ tự lớp
bus	5	2	Đối chiếu cả tên lớp và chỉ số
motorcycle	3	3	Chuẩn hóa alias annotation trước xuất
bicycle	1	4	Không giữ chỉ số COCO cũ làm ID đích

training.py kiểm tra checkpoint có đủ 80 tên lớp và các ID phương tiện mong đợi, kiểm tra SHA256 rồi gọi native train với cls_remap=True. Không đơn giản cắt năm dòng đầu của COCO vì thứ tự năm lớp đích khác nhau. Phần đầu ra phân lớp được cấu hình cho nc=5; các trọng số được chuyển theo cơ chế của package cố định. Điều này giảm nguy cơ train đúng số lớp nhưng diễn giải sai ý nghĩa chỉ số.



Hình 2. Luồng transfer learning thực tế: cùng checkpoint COCO khởi tạo độc lập cho từng policy; calibration chọn checkpoint, development báo cáo metric.

Mỗi phiên bản v0–v4 bắt đầu độc lập từ cùng byte checkpoint COCO. V4 không tiếp tục train từ best.pt của v3, và v3 không là bước nối tiếp của v2. Nếu huấn luyện nối tiếp, model sau sẽ có lịch

sử dữ liệu và ngân sách khác, khiến tác động preprocessing bị lẫn với tác động train thêm. best.pt của từng policy là sản phẩm cuối của một nhánh thí nghiệm riêng.

2.1.7. Full fine-tune và quy mô model quan sát được

freeze=0 cho phép cập nhật các khối được huấn luyện của toàn model thay vì đóng băng một số lớp backbone như nghiên cứu20epoch lịch sử. Với mọi policy, kiến trúc và khởi tạo được giữ chung; chỉ ảnh offline khác theo toán tử và gate. Full fine-tune cho phép detector thích nghi với phân bố ảnh đã biến đổi, nhưng không bảo đảm policy nào cũng tốt hơn Raw.

Bảng 6. Số tham số đọc trực tiếp từ checkpoint trên CPU

Checkpoint	Số lớp	Số tham số	Phạm vi đếm
COCO yolo26n.pt	80	2.572.280	Chưa fuse; giữ hai nhánh detection
B-Raw best.pt	5	2.535.830	Chưa fuse; giữ hai nhánh detection
Log eval nhóm A	5	2.390.707	Summary fused đã lưu trong notebook

Ba số trên thuộc các trạng thái khác nhau. Số lớp, cấu trúc head còn giữ và phép fuse Conv/BatchNorm hoặc loại nhánh không dùng có thể thay đổi số tham số báo cáo. Vì vậy không xem chênh lệch này là bằng chứng model sai phiên bản, cũng không trình bày một con số xấp xỉ2,4 triệu như thể đúng cho mọi trạng thái huấn luyện. Việc đọc metadata lần này chỉ chạy trên CPU, không train, không suy luận ảnh và không ghi đè checkpoint.

2.1.8. Quan hệ giữa model và preprocessing trong đồ án

Các policy v1–v4 nằm ngoài YOLO26n: đọc ảnh gốc, tính quality/gate, biến đổi pixel, xuất JPEG95 rồi mới đưa vào dataloader. Mạng vẫn nhận ảnh màu ba kênh. Không thêm kênh cạnh thứ tư, không thay backbone và không đưa Canny thành một loss học được. Vì vậy cách gọi phù hợp là YOLO26n được fine-tune trên ảnh tiền xử lý theo policy tương ứng.

Ảnh sáng hơn có thể thay đổi phân bố cường độ mà các bộ lọc đầu mạng nhìn thấy; tương phản cục bộ có thể làm nổi cả xe và nền; Canny có thể tập trung tăng chi tiết gần biên. Những tác động này giải thích lý do cần đo detection, nhưng không dự đoán chắc chiều biến động AP. Các kết quả chương4 là kiểm chứng thực nghiệm của các giả thuyết đó trên cùng detector trong từng nhóm cấu hình.

Đối với người đọc chưa quen model, có thể tóm tắt hệ thống như sau: preprocessing quyết định ảnh trông như thế nào; YOLO26n học cách phát hiện năm loại xe từ ảnh đó; calibration quyết định checkpoint nào được giữ; evaluator so hộp dự đoán với GT để tính AP. Bốn vai trò này được tách rõ để không nhầm độ đẹp của ảnh, mức loss, confidence và chất lượng phát hiện cuối.

2.2. Gamma và điều chỉnh chiếu sáng

Biến đổi Gamma là một ánh xạ phi tuyến theo mức sáng. Theo quy ước lũy thừa của dự án, Gamma nhỏ hơn 1 làm tăng các mức trung gian. OpenCV mô tả kiểu hiệu chỉnh phi tuyến này trong tài liệu biến đổi mức sáng [4]. Trong v1, phép biến đổi chỉ được kích hoạt cho ảnh có độ sáng toàn cục thấp; ảnh còn lại đi qua nhánh Skip.

Điều chỉnh toàn cục có thể chưa xử lý tốt bóng đổ không đồng đều. V2 vì vậy ước lượng bản đồ chiếu sáng trơn ở độ phân giải thấp và áp dụng hệ số khuếch đại bị chặn trên ảnh gốc. Đây là thiết kế cụ thể của mã nguồn dự án, không phải một mô hình ước lượng ánh sáng đã được huấn luyện hay một phương pháp Retinex được kiểm chứng độc lập [9].

2.3. CLAHE và tương phản cục bộ

CLAHE chia ảnh thành các vùng nhỏ, cân bằng histogram theo vùng và giới hạn mức khuếch đại tương phản. Tài liệu OpenCV phân biệt CLAHE với cân bằng histogram toàn cục [5]. Đối với phát hiện phương tiện, tăng tương phản mạnh có thể đồng thời làm nổi chi tiết nền hoặc nhiễu; do đó v3 sử dụng offset có giới hạn và điều kiện định tuyến, thay vì thay toàn bộ ảnh bằng đầu ra CLAHE.

2.4. Canny và giả thuyết tăng chi tiết theo biên

Canny đặt trọng tâm vào khả năng phát hiện và định vị biên [6]. Cách triển khai phổ biến gồm làm trơn, tính gradient, loại bỏ đáp ứng không cực đại và nối biên bằng hai ngưỡng hysteresis [7]. Bản đồ biên biểu diễn biến thiên cường độ, không tự phân biệt biên xe với cây, biển báo hay vạch đường.

Giả thuyết v4 là chỉ tăng chi tiết trong một lân cận mềm quanh biên Canny sẽ hữu ích hơn tăng nét toàn ảnh. Tuy nhiên, chất lượng biên không đồng nhất với AP phát hiện đối tượng. Việc ghép Canny vào pipeline phải được kiểm chứng trên detector đã huấn luyện, với cùng GT và cùng quần

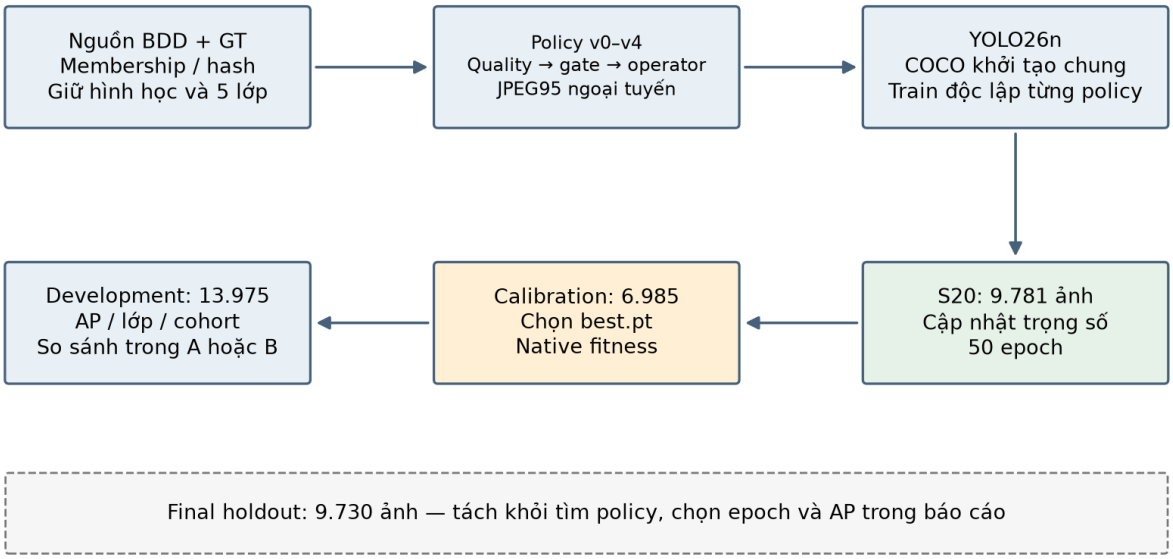
thể đánh giá. Các công thức mặt nạ, giới hạn offset và gate ở chương sau là thiết kế của dự án, không phải kết luận từ bài báo Canny rằng AP phương tiện sẽ tăng.

2.5. Khoảng trống cần kiểm chứng

Nghiên cứu cần tách ba vấn đề: ảnh nhìn rõ hơn, đặc trưng pixel thay đổi, và detector thực sự phát hiện tốt hơn. So sánh một mô hình Raw với một mô hình huấn luyện trên ảnh đã lọc chỉ phản ánh thay đổi của cả pipeline. Nó chưa tách riêng tác động tại inference khỏi tác động do huấn luyện lại hoặc chọn checkpoint trên phân bố calibration khác nhau. Một thiết kế factorial 2×2 được nêu như bước kiểm chứng tiếp theo, không được mô tả là kết quả đã đo.

CHƯƠNG 3. PHƯƠNG PHÁP NGHIÊN CỨU

Chương này trình bày phương pháp theo mức có thể tái lập: đơn vị quan sát, dữ liệu được phép sử dụng, quy tắc biến đổi ảnh, cấu hình học và cách tính chỉ số. Đối tượng so sánh là toàn pipeline gồm ảnh theo policy và detector được fine-tune tương ứng. Thí nghiệm không thay kiến trúc YOLO26n để tối ưu riêng từng policy. Mỗi kết luận phải gắn với nhóm thực thi A hoặc B đã xác định. Các chi tiết được đối chiếu trực tiếp với mã và cấu hình project[14].



Hình 3. Thiết kế phương pháp: policy tác động lên pixel trước dataloader; calibration chọn model; development đo kết quả. Final holdout được tách khỏi luồng so sánh.

3.1. BDD100K và dữ liệu sử dụng

Gói ảnh 100K của BDD100K sử dụng các khung hình tại giây thứ 10 của video và có các phân chia train/val/test chính thức [2]. Đồ án sử dụng ảnh detection và JSON nhãn, sau đó áp dụng bảng phân chia cố định của dự án. Vai trò tên là test trong data.yaml của Ultralytics ở đây là development của nghiên cứu, không phải tập test không nhãn chính thức của BDD100K.

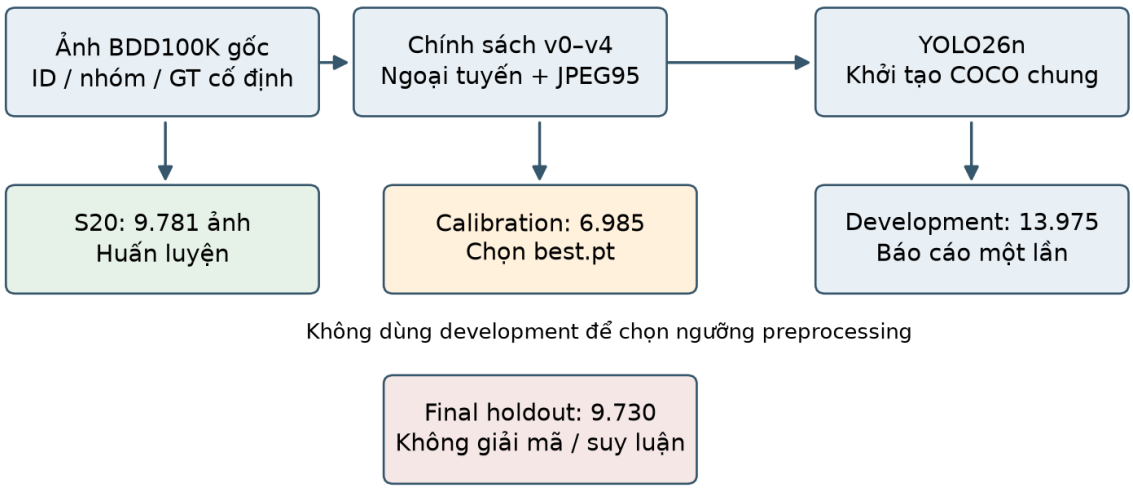
Bản chốt cục bộ có 79.593 thành viên hợp lệ trong bảng splits.csv.gz. Con số này mô tả quần thể đã cố định của nghiên cứu, không được diễn giải là tổng số ảnh của toàn bộ BDD100K. Có 69.863 ảnh thuộc nguồn official train được phân thành training pool, calibration và development; 9.730

thành viên từ official val thuộc final holdout. Báo cáo không tự gán nguyên nhân cho mọi chênh lệch giữa các số lượng này với số lượng danh nghĩa trong công bố dataset.

Bảng 7. Quần thể và nhóm của bảng chia cố định

Vai trò	Ảnh	Nhóm đã hợp nhất	Nhóm nguồn	Công dụng
Training pool	48.903	29.615	29.615	Chọn S20 để train
Calibration	6.985	4.236	4.236	Chọn checkpoint
Development	13.975	8.258	8.259	Báo cáo metric
Final holdout	9.730	5.836	5.836	Giữ kín

S10 gồm 4.890 ảnh và là tập con của S20. S20 gồm 9.781 ảnh, thuộc 5.915 nhóm. Tên S20 là định danh tập con của dự án; không nên lấy nó để suy ra rằng 9.781 là đúng 20% của một tổng bất kỳ chưa được định nghĩa. Các nhóm nguồn suy từ tên ảnh và nhóm trùng đã nhận diện được giữ nguyên khi phân vai trò; cơ chế này không bảo đảm loại hết quan hệ video hoặc ảnh gần trùng chưa nhận diện. Số nhóm nguồn và nhóm đã hợp nhất có thể khác nhau vì việc nhận diện ảnh trùng.



Hình 4. Quy trình dữ liệu và vai trò của từng tập; final holdout không tham gia chọn chính sách, checkpoint hoặc báo cáo AP.

3.1.1. Đơn vị quan sát và quy tắc sử dụng dữ liệu

Một hàng trong splits.csv.gz xác định một ảnh bằng id, vai trò, đường dẫn nguồn, kích thước, nhóm và các dấu vết hash. Đơn vị đo AP là detection/GT trong ảnh; đơn vị dùng bảo vệ chia tập là nhóm nguồn hoặc nhóm đã hợp nhất. Vì nhiều ảnh có thể liên quan cùng nhóm, không xem tổng số ảnh là số đơn vị độc lập thống kê. Khi một ảnh có nhiều xe, các hộp là nhiều instance nhưng ảnh chỉ có một tư cách thành viên của một vai trò.

Bảng 8. Quyền sử dụng từng quần thể trong nghiên cứu

Quần thể	Train trọng số	Chọn checkpoint	Báo cáo AP	Giới hạn
S20 trong training pool	Có	Không	Không dùng làm AP chính	Cùng 9.781 ID giữa policy
Calibration	Không	Có	Theo dõi hội tụ/chọn model	Không thay bằng development để chọn epoch
Development	Không	Không	Có	Phân tích mô tả sau khi policy đã cố định
Final holdout	Không	Không	Không trong báo cáo này	Không mở/giải mã/suy luận để viết báo cáo

Điểm calibration trong quá trình train là thông tin phục vụ lựa chọn mô hình; AP development là kết quả được báo cáo sau lựa chọn. Hai tập khác nhau về ID nên hai giá trị không được thay thế cho nhau. Việc xem development để viết phần thảo luận cũng phải được ghi nhận như một lần tiếp xúc với tập này; không được sau đó gọi nó là tập độc lập chưa từng được quan sát.

3.1.2. Nhận diện ảnh và annotation nguồn

scan_bdd tìm đúng một thư mục images/100k bên dưới data_root, lập bảng ảnh theo cặp official_split/tên file và đọc JSON detection theo luồng. Bộ đọc hỗ trợ JSON mảng BDD2018 hoặc cấu trúc frames của Scalabel, kiểm tra labels là danh sách và không chấp nhận ID lặp giữa các official split. Dữ liệu chỉ có ảnh nhưng thiếu JSON nhãn không đủ để train/eval theo lifecycle này.

source_rows đối chiếu bảng nguồn với membership cố định: đường dẫn, official_split, source_group và hash nhãn phải khớp. Ở các vai trò được phép đọc ảnh, audit còn kiểm tra hash byte và kích thước giải mã. Với final, hệ thống chỉ sử dụng membership/annotation metadata đã có để bảo vệ vai trò; không dùng bước kiểm tra làm lý do đọc lại pixel final. Báo cáo mô tả cơ chế từ mã nguồn, không thực hiện lại việc scan/giải mã toàn bộ dataset khi soạn thảo.

3.1.3. Điều kiện hình học và xử lý nhãn bất thường

annotation_profile kiểm tra tọa độ hữu hạn và điều kiện $0 \le x1 < x2 \le W$, $0 \le y1 < y2 \le H$. Nhãn malformed, tọa độ không hữu hạn hoặc hộp sai hình học được đánh dấu lỗi; lifecycle chuẩn bị dừng khi source_rows thấy lỗi. Mã không âm thầm cắt hộp ra biên ảnh rồi tiếp tục. Nhãn crowd/ignored bị loại theo quy tắc đã có; các hộp trùng hoàn toàn sau chuẩn hóa lớp/tọa độ được loại trùng nhất quán trước ánh xạ năm lớp.

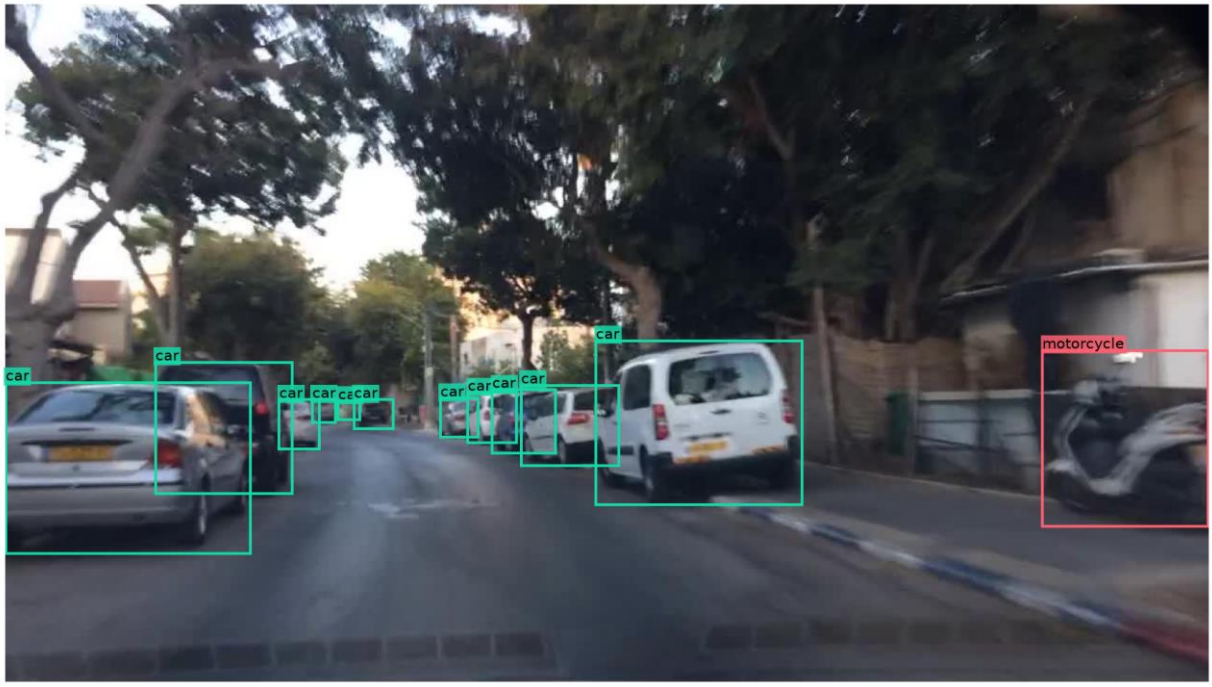
Các loại đối tượng ngoài taxonomy phương tiện không trở thành lớp thứ sáu ngầm định. Alias motor/bike được chuẩn hóa thành motorcycle/bicycle. Lớp đường sắt train bị bỏ khi đổi sáu lớp lịch sử sang năm lớp hiện tại, còn ảnh được giữ dù không còn hộp. Đây là cơ chế tạo ảnh nền hợp lệ, khác với file ảnh không đọc được hoặc annotation bị lỗi.

3.2. Nhãn năm lớp và toàn vẹn hình học

Bảng 9. Ánh xạ nhãn detection sang bài toán năm lớp

Lớp đầu ra	ID	Xử lý nhãn nguồn
car	0	Giữ lớp car
truck	1	Giữ lớp truck
bus	2	Giữ lớp bus
motorcycle	3	Chuẩn hóa bí danh motor
bicycle	4	Chuẩn hóa bí danh bike
train (đường sắt)	Loại	Không nhầm với split train; ảnh trở thành âm tính vẫn được giữ

Toán tử preprocessing không crop, quay hay đổi kích thước đầu ra nguồn. Hộp được lưu theo định dạng YOLO chuẩn hóa (class, tâm x, tâm y, rộng, cao). Ảnh không còn đối tượng thuộc năm lớp vẫn nằm trong tập với nhãn rỗng. Các kiểm tra xác minh định danh, kích thước, nhãn và hash; không thay GT để phù hợp ảnh đã tăng cường. Mọi ảnh đầu vào detector vẫn có ba kênh BGR/RGB theo luồng đọc native, không được thay bằng bản đồ cạnh đơn kênh.



Hình 5. Ví dụ calibration 05391f8a-a0e258ec với 12 hộp GT năm lớp. Đây là nhãn tham chiếu, không phải dự đoán của mô hình.

3.2.1. Ví dụ chuyển nhãn và kiểm tra sai số

Ví dụ minh họa, không lấy từ một ảnh thực nghiệm: ảnh 1280×720 có hộp car=(100,100,300,200). Nhãn YOLO là class_id=0, tâm=(200,150), rộng 200, cao 100; sau chuẩn hóa được một dòng xấp xỉ: 0 0.15625 0.208333 0.15625 0.138889. Tọa độ chuẩn hóa không có đơn vị pixel; mỗi dòng vẫn tương ứng đúng một GT của ảnh.

verify_label đọc lại từng file nhãn, kiểm tra mỗi dòng có 5 giá trị và số dòng bằng số hộp đã cố định. Tọa độ được đối chiếu bằng np.allclose với atol=1e−6, rtol=0. Đây là dung sai số học do ghi dạng thập phân, không phải quyền dịch hộp tùy ý. File nhãn rỗng hợp lệ với ảnh không có đối tượng thuộc năm lớp. Không bỏ ảnh nền để làm tỷ lệ phát hiện trông tốt hơn.

3.3. Đặc trưng chất lượng và quy ước chung

Độ sáng b, tương phản c, sharpness, ước lượng noise và tỷ lệ cực trị được tính từ ảnh gốc ở cạnh dài tối đa 512. Ảnh nhỏ hơn không bị phóng to. Việc cố định thang đo giúp ngưỡng định tuyến không phụ thuộc trực tiếp vào kích thước nguồn; riêng Canny và bản đồ chi tiết v4 được tính ở độ phân giải gốc. Tất cả các chính sách chạy trước mã hóa JPEG95 ngoại tuyến.

$$b = \text{mean}(Y_{512}) / 255; \quad c = \text{std}(Y_{512}) / 255$$

$$\text{noise} = \text{median}(|Y_{512} - \text{medianBlur}_3(Y_{512})|) / (0{,}6745 \times 255)$$

Sharpness là phương sai Laplacian. Black fraction và white fraction lần lượt là tỷ lệ pixel xám ≤3 và ≥252. Các đại lượng này là tín hiệu ảnh, không phải xác suất detector sẽ cải thiện. Skip giữ nguyên pixel nguồn trước mã hóa; v0 cũng re-encode JPEG95 nên phép so sánh không trộn lẫn lợi ích của bộ lọc với thay đổi chất lượng JPEG.

3.3.1. Từ ảnh gốc tới vector đặc trưng chất lượng

validate_image yêu cầu ndarray uint8, hình H×W×3 theo BGR và mỗi chiều ít nhất 2 pixel. quality tính scale=min(1,512/max(H,W)), resize bằng INTER_AREA rồi chuyển grayscale bằng OpenCV. Mọi đặc trưng gate thông thường lấy từ bản này; không tính lại trên output JPEG để quyết định lần hai. Việc thu nhỏ ảnh quality không thay kích thước ảnh xuất detector.

Bảng 10. Đặc trưng, đơn vị và vai trò trong gate

Đặc trưng	Cách tính / thang	Dùng trong policy
brightness b	mean(gray)/255; không thứ nguyên	v1, v3, v4; định nghĩa dark/bright
contrast c	std(gray)/255; không thứ nguyên	Gate v3
sharpness	Var(Laplacian(gray)); không chuẩn hóa /255	Có trong metadata; không điều khiển gate v2/v3/v4
noise n	Median residual tuyệt đối/(0,6745×255)	Gate v3/v4; chỉ là ước lượng từ pixel
black_fraction	Tỷ lệ gray≤3 ở thang 512	Gate v3
white_fraction	Tỷ lệ gray≥252 ở thang 512	Gate v3
edge_density d	Số pixel Canny khác0/(H×W) ở ảnh nguồn	Gate v4; đo trước dilation

Noise ở đây là thống kê residual so với medianBlur3, không phải phép đo nhiễu cảm biến có ground truth. Sharpness phụ thuộc thang và toán tử Laplacian; không so trực tiếp với một giá trị từ ảnh nguyên độ phân giải hoặc kernel khác. Ngưỡng Canny50/150 là ngưỡng đáp ứng gradient trong cách triển khai OpenCV, không phải hai mức brightness50/255 và150/255.

3.3.2. Biên ngưỡng và ví dụ quyết định Apply/Skip

Bảng 11. Các tình huống biên minh họa theo toán tử so sánh trong mã

Tình huống giả định	Quyết định	Lý do
v1: b=0,1200, noise≤1	Skip	Gamma dùng b<0,12; nhánh khác không kích hoạt
v3: c=0,2000	Skip	contrast phải nhỏ hơn0,20
v3: b=0,04 hoặc0,80	Skip	Cả hai biên brightness đều chặt
v3: noise=0,01 và điều kiện khác đạt	Apply	noise dùng ≤, không dùng <
v4: d=0,001 hoặc0,20	Skip	Edge density nằm trong khoảng mở
v4: noise=0,02, b/d hợp lệ	Apply	Noise ở trần vẫn được chấp nhận

Gate là phép AND của các điều kiện: một điều kiện không đạt thì Skip. Ví dụ một ảnh có b=0,10, c=0,15, noise=0,005, black=0,10 và white=0,01 sẽ đạt gate v3; nếu chỉ đổi noise thành0,02 thì không đạt. Đây là tình huống số học minh họa quy tắc, không phải một mẫu đã được đo trong BDD100K. Gate dùng thông tin pixel, không đọc nhãn GT, confidence hoặc AP để quyết định ảnh nào được xử lý.

3.4. Các chính sách v0–v4

Bảng 12. Tổng hợp chính sách đã triển khai

Phiên bản	Cơ chế	Điều kiện / giới hạn chính
v0	Raw / Skip	Không tăng cường; JPEG95
v1	Gamma-or-Skip	b < 0,12; Gamma = 0,85
v2	Local-shadow	Bản đồ Gaussian σ16 ở thang 512; gain 1–2
v3	Adaptive local contrast	CLAHE cục bộ; offset mỗi kênh ≤12; gate nhiều đặc trưng
v4	Canny-guided detail	Canny 50/150; mặt nạ mềm; offset mỗi kênh ≤12; có Skip

3.4.1. V0 — đối chứng Raw

V0 không thực hiện toán tử tăng cường. Cùng nguồn ảnh, cùng nhãn và cùng JPEG95 được dùng như các chính sách còn lại. Đối chứng này cho biết mức AP có thể đạt khi detector được fine-tune mà không thay đổi chiếu sáng, tương phản hay chi tiết biên.

3.4.2. V1 — Gamma-or-Skip

$$I' = \text{floor}[255 \times (I / 255)^{0,85}] \text{ nếu } b < 0,12; \text{ ngược lại } I' = I$$

Ánh xạ được cài bằng LUT cho mỗi kênh uint8. Ngưỡng brightness được đo trên ảnh gốc. Đây là chính sách lịch sử có tên thuật toán "Adaptive v2", nhưng định danh thư mục thí nghiệm hiện tại là v1; không đồng nhất nó với v2 Local-shadow [9]. Nhánh Skip tránh chỉnh những ảnh không thuộc miền tối được khai báo, không phải một bộ chọn dựa trên GT.

3.4.3. V2 — Local-shadow

$$L = \text{Gaussian}_{16}(\text{gray}(I_{512})) / 255$$

$$g = \text{clip}(\sqrt{[0,25 / \max(L, 0,0625)]}, 1, 2); \quad I' = \text{clip}(I \times \text{resize}(g), 0, 255)$$

Bản đồ gain được phóng về kích thước gốc bằng nội suy tuyến tính và nhân đều vào các kênh. Gain không vượt 2; chi tiết nguồn không bị thay bằng ảnh đã làm trơn. V2 áp dụng toán tử cho mọi ảnh, vì vậy không có một cohort Skip độc lập để mặc định so sánh. Khuếch đại vùng tối cũng có thể khuếch đại nhiễu; điều này là giả thuyết rủi ro cần đo, không phải kết quả đã suy ra từ ảnh minh họa.

3.4.4. V3 — tương phản cục bộ thích nghi

V3 chỉ Apply khi $c<0,20$; $0,04<b<0,80$; noise$\leq 0,01$; black fraction$<0,30$ và white fraction$<0,10$. Các ảnh còn lại Skip. CLAHE clip1,5 với lưới 8×8 tạo ảnh xám E; phần khác biệt với Y được đưa về offset có giới hạn, có suy giảm ở gần vùng đen hoặc trắng.

$$p = \text{clip}(\min(Y - 8, 247 - Y) / 32, 0, 1)$$

$$\delta = \text{round}[\text{clip}(0,5(E - Y), -12, 12) \times p]; \quad I' = \text{clip}(I + \delta, 0, 255)$$

Offset được cộng đều vào các kênh BGR nhưng clipping ở kênh bão hòa vẫn có thể làm lệch màu. Giới hạn 12 là trên pixel trước mã hóa JPEG; không phải một bảo đảm rằng mọi sai khác giữa hai ảnh JPEG đã giải mã đều ≤12. V3 là một chính sách định tuyến và tăng cường cụ thể, không thay đổi nhãn lớp hoặc tọa độ hộp.

3.4.5. V4 — tăng chi tiết theo mặt nạ Canny

Ảnh xám gốc được làm trơn Gaussian σ1,0. Canny dùng ngưỡng 50 và 150, aperture3 và L2gradient=True. Edge density d là tỷ lệ pixel biên trước dilation. V4 Apply khi $0,04<b<0,90$,

noise≤0,02 và 0,001<d<0,20; nếu không thì Skip. Biên được giãn với kernel3×3 rồi làm mềm Gaussian3×3, σ0,8 thành mặt nạ M trong [0,1].

$$D = Y - \text{Gaussian}_{\text{I}}(Y)$$

$$\delta = \text{round}[\text{clip}(0{,}5D, -12, 12) \times M \times p]; \quad I' = \text{clip}(I + \delta, 0, 255)$$

Hệ số bảo vệ p giống v3. Vì mặt nạ có hỗ trợ hữu hạn, pixel ngoài lân cận biên không bị đổi bởi toán tử. Giả thuyết là tập trung chi tiết gần biên giúp detector mà không tăng nét mọi nền ảnh. Canny không nhận biết xe theo ngữ nghĩa; biên do cảnh nền vẫn có thể được tăng cường. Công thức giới hạn thay đổi và gate là phần thiết kế của dự án [9].



Hình 6. Ảnh calibration 6bbb63a9-5d1b6029 qua v0–v4, chọn từ mẫu seed2026 theo độ sáng, không theo kết quả detector. Hình là preview pixel trước JPEG95, không phải bằng chứng tăng AP.



Hình 7. Chi tiết v4 trên cùng ảnh calibration: bản đồ Canny chỉ dùng để hướng dẫn mặt nạ; detector nhận ảnh màu. Thang thay đổi 0–12 áp dụng trước JPEG.

3.4.6. Giả mã thao tác của từng phiên bản

Ký hiệu I là ảnh BGR uint8 nguồn, Y là grayscale ở độ phân giải nguồn, Q là quality từ thang 512 và J là kết quả trước JPEG. Một phiên bản chỉ thực hiện policy được khai báo; không chạy lần lượt v1→v2→v3→v4. Mỗi lần xuất đều đọc lại cùng ảnh gốc. Các bước dưới đây mô tả thứ tự thực thi, bổ sung cho công thức ở các mục 3.4.1–3.4.5.

Bảng 13. Giả mã v0 và v1

Policy	Bước	Thao tác
v0	1	Đọc I, xác minh dtype/kích thước/hash; đo Q để ghi metadata
v0	2	Đặt J=I, applied rỗng; không chạy toán tử tăng cường
v0	3	Mã hóa J thành JPEG95 theo cùng quy ước export
v1	1	Tính Q trên I và các điều kiện wanted của variant adaptive
v1	2	Chọn tối đa1 nhánh theo thứ tự gamma/clahe/denoise/sharpen; không có nhánh thì J=I
v1	3	Trong export đã đối chiếu chỉ có gamma hoặc Skip; Gamma dùng LUT floor(255×(k/255)^0,85)
v1	4	Tra LUT trên B/G/R khi chọn Gamma; xuất JPEG95 và nhãn giữ hình học

V1 dùng variant=adaptive với brightness_below=0,12, contrast_below=0, noise_above=1, sharpness_min=sharpness_below=0 và max_filters=1. Metadata export_record của B-v1 ghi7.093 ảnh gamma và23.648 ảnh Skip trên tổng30.741 ảnh, không ghi toán tử khác. Do đó Gamma-or-Skip là mô tả đúng export được báo cáo. Tuy nhiên noise là ước lượng chuẩn hóa, không được bảo đảm luôn≤1 trên mọi ảnh nhân tạo; không khẳng định ngưỡng noise_above=1 làm nhánh denoise bất khả thi về toán học. Việc có cấu hình một toán tử trong YAML không chứng minh nó đã chạy; cần kiểm tra applied trên quần thể thực tế.

Bảng 14. Giả mã v2 — bản đồ gain theo vị trí

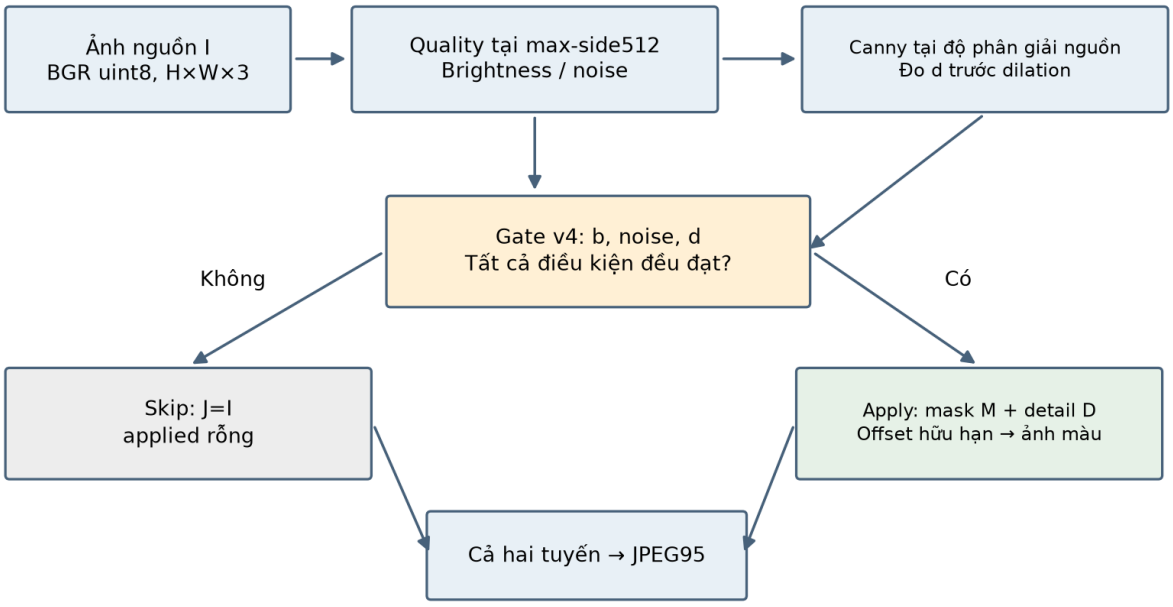
Bước	Thao tác	Chi tiết cần giữ
1	Thu nhỏ I về cạnh dài≤512 và đổi grayscale	INTER_AREA; không phóng ảnh nhỏ
2	GaussianBlur(gray,(0,0),σ=16)	σ ở thang 512; không làm trơn ảnh đầu ra trực tiếp
3	Chuẩn hóa illumination và tính gain	sqrt(0,25/max(L,0,0625)); clip[1,2]
4	Resize gain về(W,H)	INTER_LINEAR; áp chung một gain cho cả B/G/R
5	Nhân I.float32×gain; clip[0,255]	Sau đó astype(uint8); không dùng làm tròn np.rint
6	Ghi applied=local_shadow, xuất JPEG95	Luôn đi qua toán tử; không có gate Skip theo ảnh

Bảng 15. Giả mã v3 — offset CLAHE có bảo vệ

Bước	Thao tác	Chi tiết cần giữ
1	Tính Q và kiểm tra đủ gate v3	Skip ngay nếu bất kỳ điều kiện nào không đạt
2	Đổi grayscale nguồn; CLAHE clip 1,5, grid 8×8	Tạo E trên grayscale, không CLAHE riêng từng kênh BGR
3	Đổi Y sang float32 và tính p	p=clip(min(Y−8,247−Y)/32,0,1)
4	Clip 0,5×(E−Y) trong[−12,12]	Thực hiện trước nhân p
5	Offset=np.rint(offset_clipped×p)	Làm tròn theo NumPy; dùng kiểu signed khi cộng
6	J=clip(I.int16+offset.int16,0,255).uint8	Cùng offset cho ba kênh; sau đó mới JPEG95

Thứ tự ép kiểu là một phần của phương pháp: tính E−Y với uint8 có thể gây underflow, còn cộng offset âm vào uint8 có thể gây wrap-around. Mã thực tế chuyển về float32/int16 trước các bước

tương ứng. np.rint làm tròn về số nguyên gần nhất, với trường hợp nửa đơn vị theo quy tắc của NumPy; đây là quy ước khác với cắt phần thập phân của astype(uint8) ở v2.



Hình 8. Luồng v4 theo apply_recipe: Canny được tính để đo gate cả khi ảnh cuối cùng Skip; biên được tái sử dụng khi Apply.

Bảng 16. Giả mã v4 — Canny-guided detail

Bước	Thao tác	Tham số / điều kiện
1	Đo Q từ ảnh gốc ở thang 512	Brightness và noise cho gate
2	Gray nguồn→Gaussian→Canny	σ=1; threshold50/150; aperture3; L2=True
3	Đo d từ cạnh nhị phân trước dilation	d=mean(edges!=0); kiểm tra gate v4
4	Nếu Skip: J=I và applied rỗng	Canny vẫn đã được tính ở bước2
5	Nếu Apply: dilation và làm mềm mask	Kernel3×3; Gaussian 3×3, σ=0,8; chia255
6	D=Y−Gaussianσ1(Y.float32)	Bản đồ chi tiết, khác bản đồ edges nhị phân
7	δ=rint(clip(0,5D,−12,12)×M×p)	p bảo vệ vùng đen/trắng như v3
8	Cộng δ bằng kiểu signed, clip rồi uint8	Ảnh màu ba kênh, giữ tọa độ, JPEG95 sau cùng

3.4.7. Ý nghĩa và giới hạn của ràng buộc pixel

V3/v4 giới hạn offset trước cộng ảnh trong ±12 mức; clipping ở[0,255] không làm độ thay đổi tăng thêm. Bảo vệ p và mask M chỉ giảm biên độ. Tuy nhiên sau JPEG, chênh lệch giữa hai ảnh

giải mã còn phụ thuộc mã hóa nên không dùng trần12 làm assertion cho toàn bộ pixel JPEG. Với v4, một pixel ngoài hỗ trợ hữu hạn của mask giữ nguyên trước JPEG; điều đó không đồng nghĩa cả khối JPEG chứa pixel đó giữ nguyên byte.

Skip có nghĩa không tăng cường pixel trước export, không có nghĩa sao chép nguyên byte file JPEG nguồn. Raw cũng được re-encode JPEG95 để các nhánh có cùng một bước mã hóa. Nhờ vậy, so sánh không trộn biến đổi quang học với việc một policy dùng JPEG nguồn và policy khác chịu nén lại. Tính xác định của thuật toán không tự bảo đảm mọi phiên bản OpenCV cho output byte giống nhau; cần giữ runtime và hash.

3.5. Cấu hình huấn luyện và chọn checkpoint

Máy chạy dùng NVIDIA GeForce GTX 1660 Ti 6 GiB. Các lượt đối chiếu mới sử dụng batch16 và workers0 để phù hợp tài nguyên, thay cho batch32/workers8 trong cấu hình canonical v0–v3. Các alias như v5_raw và v5_v1 là lượt chạy đối chứng đồng bộ của chính sách v0/v1; chúng không biến phương pháp Raw thành thuật toán v5. Tên run được giữ trong phụ lục để tránh nhầm nguồn số liệu.

Bảng 17. Recipe native chung của các lượt so sánh đồng bộ

Tham số	**Giá trị**
Mô hình / khởi tạo	YOLO26n; COCO yolo26n.pt, SHA256 cố định
Seed / epoch	2026 / 50; patience=0
Fine-tune / optimizer	freeze=0; SGD; lr0=0,01; momentum=0,937; weight_decay=0,0005
Kích thước / batch / worker	640 / 16 / 0; cache=False; AMP=True
Gradient accumulation chuẩn	nbs=64; giữ chung giữa các lượt
Augmentation hình học	mosaic=1,0; close_mosaic=10; fliplr=0,5; scale=0,5; translate=0,1
Augmentation quang học	hsv_h=hsv_s=hsv_v=0; bgr=0
Checkpoint	best.pt theo native fitness trên calibration của từng chính sách
Runtime thực tế	PyTorch 2.7.1+cu126; Ultralytics 8.4.198; OpenCV 5.0.0

Không khởi tạo v4 từ mô hình Raw đã fine-tune. Tất cả bắt đầu độc lập từ cùng byte COCO. Calibration được xử lý theo cùng chính sách của phiên bản đó để chọn checkpoint; development không dùng chọn epoch. Runtime OpenCV 5.0.0 thực tế phải được ghi nhận khi tái lập: nó khác

ràng buộc opencv-python<5 trong pyproject.toml. Các lượt đã hoàn tất dùng cùng runtime, nhưng không thể mô tả môi trường này là một bản uv sync --locked hoàn chỉnh.

3.5.1. Quy tắc chọn checkpoint và kiểm soát ngân sách

Trong runtime Ultralytics cục bộ đã kiểm tra, Metric.fitness dùng trọng số[0,0,0,1] cho[P,R,mAP50,mAP50:95], tức fitness là mAP@50:95 trên calibration. Vì thế best.pt được chọn theo thước đo chính này, dù người thực hiện còn quan tâm mAP@50. Nếu chọn lại epoch có AP50 development cao nhất, đó sẽ là một quy trình nghiên cứu khác và làm development tham gia chọn model.

patience=0 tắt cơ chế early stopping theo cấu hình, để budget mục tiêu là50 epoch cho mỗi lần full fine-tune. last.pt biểu thị checkpoint gần cuối của tiến trình; best.pt không bắt buộc là epoch cuối. Trong báo cáo, epoch đỉnh lấy từ đường calibration/log là thông tin mô tả; để xác minh độc lập checkpoint cần artifact và hash, không chỉ nhìn vị trí cực đại đã làm tròn trên biểu đồ.

Một policy có cùng số epoch nhưng khác batch hoặc thay learning-rate schedule chưa phải đối chứng hoàn toàn phù hợp. NhómA giữ batch32/workers8, nhómB giữ batch16/workers0; augmentation và checkpoint khởi tạo được giữ theo recipe của từng nhóm. Các alias v5_raw/v5_v1 là tên run của đối chứng mới, không phải thay kiến trúc detector hoặc đổi ý nghĩa thuật toán Raw/Gamma.

3.6. Thước đo đánh giá

$$\text{IoU}(A, B) = \text{diện_tích}(A \cap B) / \text{diện_tích}(A \cup B)$$

$$\text{Precision} = TP / (TP + FP); \quad \text{Recall} = TP / (TP + FN)$$

AP là diện tích dưới đường precision–recall theo lớp và ngưỡng IoU. mAP@50 lấy trung bình AP trên năm lớp tại IoU=0,50; mAP@50:95 lấy trung bình trên các ngưỡng 0,50 đến 0,95, bước 0,05 [8]. Dự án dùng cách tính native Ultralytics với lớp và quần thể đã xác định; không tự khẳng định hoàn toàn tương đương mọi chi tiết của COCOeval. mAP không phải tỷ lệ ảnh dự đoán đúng.

Cấu hình validation là FP32, imgsz640, batch16, conf0,001, IoU0,7, max_det300, rect=True, augment=False, nms=False. mAP@50:95 là thước đo chính của dự án; mAP@50 được báo cáo đồng thời theo mục tiêu người thực hiện. Delta được tính bằng điểm phần trăm:

100×(mAP_phương_pháp − mAP_Raw), không phải phần trăm thay đổi tương đối. Precision/recall native không được coi là đã cố định cùng một ngưỡng confidence vận hành ngoài thực tế.

3.6.1. Ghép prediction với GT tại nhiều ngưỡng IoU

Evaluator xử lý từng ảnh, so các hộp dự đoán với GT cùng lớp và cùng ID ảnh. Với runtime cục bộ đã đọc, BaseValidator.match_predictions nhận prediction theo confidence giảm dần; tại mỗi ngưỡng IoU, một prediction được ghép với GT cùng lớp chưa nhận ghép có IoU cao nhất nếu đủ ngưỡng. Một GT chỉ được nhận một prediction tại ngưỡng đó. Mã có tùy chọn Hungarian, nhưng không mặc định giả định tùy chọn này đã được bật trong thí nghiệm.

$$T = \{0{,}50;\ 0{,}55;\ 0{,}60;\ \ldots;\ 0{,}95\};\quad \text{TP: } N_{pred} \text{ hàng} \times 10 \text{ cột boolean}$$

Bảng 18. Ví dụ ghép tại IoU 0,50: một lớp, hai GT trong cùng ảnh

Prediction giả định	Confidence	Quan hệ GT	Kết quả
p1	0,90	Khớp g1, IoU 0,80	TP
p2	0,80	Trùng g1, IoU 0,75; g1 đã nhận p1	FP
p3	0,70	Khớp g2, IoU 0,55	TP

Ở ví dụ trên, sau ba prediction: precision=2/3 và recall=2/2 tại IoU 0,50. Khi yêu cầu IoU 0,75, p3 không đạt nên recall không còn bằng1. Ví dụ chỉ minh họa matching, không phải bộ prediction đã quan sát trong project và không được dùng làm AP của BDD. Một hộp sai lớp là prediction không khớp GT của lớp đó, dù tọa độ trùng với một xe thuộc lớp khác.

3.6.2. Từ đường precision–recall tới AP và mAP

Sau matching theo ảnh, ap_per_class gộp detection theo lớp và xếp confidence giảm dần. TP/FP được cộng dồn để tạo đường precision–recall tại từng ngưỡng IoU. compute_ap của package cục bộ thêm sentinel, tạo precision envelope, nội suy trên101 mốc recall và tính diện tích bằng phép tích phân số. Báo cáo dùng implementation native này, không thay bằng tỷ lệ ảnh đúng hoặc một giá trị precision tại confidence cố định.

$$AP_c(t) = \text{diện tích dưới đường precision–recall của lớp } c \text{ tại IoU}=t$$

$$mAP@50 = (1/5)\cdot\Sigma_c\ AP_c(0{,}50)$$

$$mAP@50:95 = (1/50)\cdot\Sigma_c \Sigma_{t\in T} AP_c(t)$$

Công thức cuối áp dụng cho evaluation tổng thể có đủ năm lớp, mỗi lớp trọng số bằng nhau. AP có thể ở thang0–1 trong JSON và được nhân 100 khi xuất bảng phần trăm. Ví dụ0,421348 tương ứng42,1348%; delta0,000999 tương ứng0,0999 điểm phần trăm. Không gọi delta này là mức tăng0,0999% tương đối vì mẫu số và ý nghĩa khác nhau.

3.6.3. Phân biệt ba loại ngưỡng khi đọc kết quả

Bảng 19. Các ngưỡng không thể thay thế cho nhau

Thiết lập	Vai trò	Điều không được suy ra
conf=0,001	Lọc điểm đầu vào khi tạo tập detection đánh giá	Không phải confidence vận hành đã chọn cho camera thực
iou=0,7 trong args	Tham số hậu xử lý giữ chung; ý nghĩa phụ thuộc nhánh/backend	Không phải IoU dùng tính mAP@50
IoU 0,50…0,95	Chuẩn xác định matching TP tại mỗi mức	Không phải ngưỡng brightness hoặc Canny
Điểm P/R native	Chọn trên đường F1 theo implementation	Không mặc định là P/R tại conf 0,25 cho mọi model

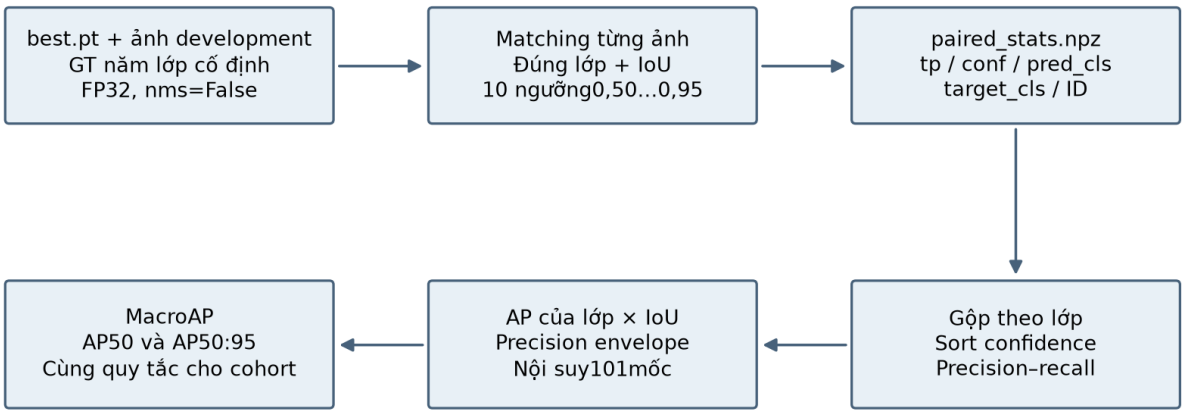
ap_per_class lựa chọn chỉ số confidence theo cực đại đường mean-F1 đã làm trơn để xuất P/R native. Vì mỗi detector có đường score khác nhau, so P/R tổng thể không đồng nghĩa so hai detector ở cùng một confidence vận hành. Nếu cần ứng dụng thực tế, phải chốt ngưỡng trên calibration và báo cáo TP/FP/FN tại ngưỡng đó; phần này chưa được thực hiện mới cho báo cáo.

3.6.4. Cohort và thống kê ghép được lưu

MatchingStats lưu paired_stats.npz gồm id ảnh và các mảng tp, conf, pred_cls, target_cls. Cohort được tính bằng cách chọn lại cùng ID trong thống kê đã có rồi gọi class_ap; không chạy lại detector với ngưỡng được tối ưu riêng cho từng cohort. Dark/bright dùng brightness của ảnh nguồn trong metadata, nên membership không thay đổi chỉ vì output policy sáng hơn.

class_ap chỉ tổng hợp những lớp thực sự có GT trong phần được chọn; nếu không có GT nào, trả tập kết quả rỗng. cohort_metrics dùng trung bình trên các lớp có hỗ trợ hoặc trả null, không tự thêm lớp vắng với AP=0. Do đó AP của một cohort thiếu bicycle không tương đương macroAP

trên đủ năm lớp. Trong báo cáo, so tối/còn lại giữa policy chỉ có ý nghĩa khi giữ cùng ID và kiểm tra hỗ trợ lớp.



Hình 9. Quy trình đánh giá native: matching từng ảnh và từng IoU, xếp confidence theo lớp, tính AP rồi tổng hợp. Cùng thống kê phục vụ phân tích cohort.

3.7. Quy trình notebook và toàn vẹn chứng cứ

Bốn pha độc lập là 01_experiments (preview calibration), 02_train_prepare (xuất offline), 03_train (native training) và 04_eval (development). Mỗi pha đọc prerequisite đã lưu, không ngầm chạy lại notebook trước. Các công việc nặng cần switch rõ ràng. Ảnh gốc và checkpoint được kiểm tra hash; metadata, cấu hình, args.yaml, results.csv, best.pt và metrics.json giữ dấu vết của lượt chạy.

Nguồn A dùng bảng và log đã nhúng trong notebook, không yêu cầu có thư mục runs gốc mới được ghi nhận kết quả. Nguồn B được kiểm tra marker hoàn tất, metrics.json và hash best.pt. Không chạy inference mới để tạo số liệu cho báo cáo; hình chính sách chỉ dùng calibration. Không mở lại, giải mã hoặc đánh giá ảnh final trong quy trình viết báo cáo.

3.7.1. Điều kiện hoàn tất và các trạng thái của lifecycle

Bảng 20. Trạng thái thực nghiệm và chứng cứ cần có

Trạng thái	Dấu hiệu	Ý nghĩa trong báo cáo
Chưa chuẩn bị	Chưa có export đã verify	Không dùng data.tmp làm tập train hợp lệ

Trạng thái	Dấu hiệu	Ý nghĩa trong báo cáo
Đã chuẩn bị	data.yaml, metadata và inventories/hash khớp	Có thể bắt đầu train đúng run
Đang train	Có args/results/last, chưa có complete.json	Không coi epoch đang dở là kết quả cuối
Train hoàn tất	Marker, đủ history, best.pt khớp hash	Có thể đánh giá best.pt đúng identity
Eval hoàn tất	metrics.json và inputs khớp checkpoint/data/settings	Có thể đưa AP vào snapshot kết quả
Chứng cứ notebook A	Output bảng/log/hình được lưu	Dùng đúng mức kiểm chứng; không giả có checkpoint máy gốc

prepare_data xuất vào thư mục data.tmp và chỉ đổi tên thành data khi các bước thành công; lỗi trước đó không được coi là một export hoàn chỉnh. Khi đích đã tồn tại, hàm kiểm tra lại thay vì tự xử lý chồng lên ảnh processed. verify_data yêu cầu danh sách ID ảnh/nhãn đúng membership, không chấp nhận symlink/thư mục lạ và kiểm tra digest byte ảnh/nhãn cùng hash metadata.

Sau train, complete.json ghi epochs_run, hash best.pt và nguồn selection calibration. completed_training kiểm tra args.yaml đúng setting, hash best.pt và số dòng results.csv khớp marker. Khi eval cũ tồn tại, evaluate_experiment chỉ tái dùng nếu identity gồm run, files_sha256, checkpoint_sha256, settings và phiên bản Ultralytics trùng. File metrics có sẵn nhưng khác identity không phải kết quả có thể ghép tùy ý vào run mới.

3.7.2. Metadata nào thực sự được lưu?

Bảng 21. Các trường xuất trong metadata.csv

Trường	Nội dung	Lưu ý
id / split	Ảnh và train/val/test của export	test ở đây là development
source_group	Nhóm nguồn từ membership	Không phải dự đoán video ID mới
applied	Tên toán tử được chọn, hoặc chuỗi rỗng	Chuỗi rỗng là Skip; không là lỗi đọc ảnh
brightness / contrast	Quality ảnh nguồn ở thang 512	Được dùng chọn cohort/gate
sharpness / noise	Thống kê Laplacian và residual	Không phải độ chính xác model
black_fraction / white_fraction	Tỷ lệ mức xám cực trị	Dùng gate v3

Điểm cần lưu ý về khả năng tái kiểm tra: v4 tính edge_density trong scores để định tuyến, nhưng _export hiện chỉ ghi sáu FEATURES quality vào metadata.csv, chưa ghi edge_density từng ảnh. Vì vậy có thể kiểm tra tỷ lệ Apply từ metadata, còn kiểm tra lại ngưỡng d của từng ảnh cần source/config tương ứng và tính lại đúng toán tử. Báo cáo không trình bày một cột edge_density như thể nó đã được lưu sẵn.

operator_sha256 được ghi lúc chuẩn bị, nhưng verify_data hiện đưa khóa này vào nhóm ignored khi so cấu hình data.yaml và không tự kiểm tra lại hash mã hiện tại bằng khóa đó. Điều này cần được phân biệt với kiểm tra digest ảnh/nhãn đã xuất, vốn được thực hiện. Để truy vết báo cáo, methodology_evidence.json ghi hash mã được đọc; việc bổ sung kiểm tra operator tự động vào lifecycle là đề xuất kỹ thuật, chưa phải thay đổi đã triển khai trong lần viết tài liệu.

3.8. Hai nhóm full fine-tune và nguồn chứng cứ

Để khai thác đầy đủ kết quả đã có trong project, nghiên cứu sử dụng hai nhóm thực nghiệm A và B. Nhóm A là output đã lưu ngay trong notebook v0–v3 của lần chạy 09/11/2026. Nhóm B là các lần chạy cục bộ mới, trong đó Raw và Gamma được tái chạy theo recipe của v4. Đây là hai nhóm có cùng nhiệm vụ nhưng không phải cùng điều kiện tính toán. Ký hiệu A-v2 và B-v2 vì vậy chỉ cùng chính sách, không chỉ cùng một checkpoint.

Bảng 22. Phân biệt nhóm thực nghiệm và mức độ kiểm chứng

Thuộc tính	**Nhóm A: notebook đã lưu**	**Nhóm B: chạy cục bộ**
Phiên bản	v0, v1, v2, v3	v0, v1, v4 đã hoàn tất trong snapshot
Thời điểm / batch	09/11/2026; batch32	10–12/11/2026; batch16
Workers train / eval	8 / 0	0 / 0
GPU theo log	RTX 5060 Ti, 16.284 MiB	GTX 1660 Ti, 6 GiB
Python / PyTorch	3.11.14 / 2.15.0+cu132 theo log	3.13.9 / 2.7.1+cu126 theo runtime
Ultralytics	8.4.198	8.4.198
Chung	S20=9.781; 50 epoch; SGD; freeze=0; seed2026; imgsz640	Cùng recipe nền; khác batch, worker và runtime
Chứng cứ	Bảng HTML/text, stream log, hình PNG nhúng notebook	complete.json, metrics.json, results.csv, best.pt và hash
Giới hạn	Không có checkpoint gốc để kiểm tra lại hash trên máy này	Có checkpoint; chưa có đủ v2/v3 nhóm B

Các giá trị Python/PyTorch/GPU của nhóm A được chép đúng chuỗi trong log nhúng; báo cáo không tự xác nhận lại môi trường máy gốc. Sáu chữ số thập phân trong DataFrame là độ chính xác của chứng cứ đã lưu, không phải toàn bộ độ chính xác float bên trong evaluator. Bản báo cáo xuất bảng phần trăm với tối đa bốn chữ số, tính delta từ giá trị đã trích; chữ số cuối có thể chịu sai số làm tròn của nguồn.

Việc nbs=64 giống nhau không làm batch16 và batch32 trở thành hai recipe đồng nhất: phân hoạch mini-batch, số bước và hành vi framework có thể khác. Do đó chỉ dùng A-v2 trừ A-v0, A-v3 trừ A-v0 và B-v4 trừ B-v0 cho kết luận chính sách. Bảng tổng hợp mọi phiên bản là bản kiểm kê chứng cứ, không phải bảng xếp hạng nhân quả giữa năm phương pháp.

3.9. Cơ chế, giả thuyết và điểm nhạy của từng toán tử

Bảng 23. Giả thuyết cải thiện và kiểm tra có thể bác bỏ

Chính sách	Giả thuyết	Điểm cần kiểm chứng
v1 Gamma	Làm rõ mức sáng trung gian trong ảnh tối	AP nhóm tối có tăng không; hai lớp xe hai bánh có suy giảm không
v2 Local-shadow	Tăng vùng bóng đổ qua gain không gian trơn	Toàn ảnh Apply có làm giảm tương phản hữu ích hoặc khuếch đại nhiễu không
v3 CLAHE có gate	Tăng tương phản có giới hạn cho ảnh đủ điều kiện	AP50 tăng có đi kèm AP75/AP50:95; lợi ích có chỉ nằm ở nhóm tối không
v4 Canny	Giữ tăng chi tiết gần biên và hạn chế vùng phẳng	Biên nền có lấn tín hiệu xe; hai lớp hiếm và tổng AP có tốt hơn Raw không

3.9.1. V2: diễn giải gain theo mức chiếu sáng

V2 dùng Gaussian để tách một thành phần sáng thay đổi chậm, sau đó suy ra hệ số nhân theo vị trí. Đây là phép xấp xỉ chiếu sáng bằng bộ lọc trơn trong dự án; không phải ước lượng vật lý đã được hiệu chuẩn và không phải mô hình học sâu Retinex. Ảnh thu nhỏ chỉ phục vụ tính gain; ảnh đầu ra vẫn lấy chi tiết từ các kênh ảnh gốc.

Bảng 24. Các mốc của hàm gain v2 trước nội suy

L chuẩn hóa	Gain lý thuyết sau clip	Ý nghĩa
$L \leq 0{,}0625$	2,000	Chặn mức nhân tối đa
0,10	≈1,581	Nâng vùng tối

L chuẩn hóa	Gain lý thuyết sau clip	Ý nghĩa
0,16	1,250	Nâng vừa
L ≥ 0,25	1,000	Không hạ sáng vùng sáng

Từ công thức, gain luôn ≥1 nên toán tử không chủ động làm tối các vùng sáng. Tuy nhiên việc nhân rồi clip có thể làm kênh gần bão hòa mất chi tiết. Bản đồ Gaussian được tính trên thang cạnh dài512, vì vậy σ16 thuộc thang này; thay bằng σ16 trên ảnh nguồn sẽ tạo thuật toán khác. Apply=100% chỉ có nghĩa mọi ảnh đi qua toán tử, không có nghĩa mọi pixel đều đổi hoặc tất cả ảnh tăng sáng như nhau.

3.9.2. V3: bảo vệ mức sáng và offset hữu hạn

Khác với v2 chỉ nhân gain, v3 có thể tạo offset dương lẫn âm. CLAHE tạo một đề xuất ảnh xám, còn toán tử thực tế chỉ đưa một phần độ chênh đó vào ảnh màu. Hệ số 0,5 giảm cường độ thay đổi; clip ±12 đặt trần trước làm tròn; hệ số bảo vệ p giảm tác động gần đen và trắng. Với Y=8 hoặc Y=247 thì p=0; tại vùng 40≤Y≤215 thì p=1. Gate được quyết định một lần trên ảnh gốc, tránh vòng lặp xử lý rồi đo lại để tự thay đổi quyết định.

ClipLimit=1,5 và tileGridSize=(8,8) là hai tham số khác nhau. Lưới 8×8 là số ô theo hai chiều ảnh, không phải kernel 8 pixel. Cộng cùng offset vào BGR nhằm giữ biến thiên cường độ gần trung tính, nhưng không bảo đảm bảo toàn màu tuyệt đối khi từng kênh clip ở biên 0/255. Đây cũng là lý do không mô tả phương pháp như một phép cân bằng histogram độc lập cho từng kênh màu.

3.9.3. V4: vai trò và giới hạn của biên Canny

Ngưỡng thấp/cao 50/150 phục vụ hysteresis của Canny: biên yếu cần liên kết với biên mạnh để được giữ. Gaussian trước Canny giảm đáp ứng với chi tiết nhiễu; dilation mở rộng vùng có thể tăng cường, Gaussian sau dilation làm chuyển tiếp mặt nạ mềm hơn. Bản đồ chi tiết D lấy từ ảnh xám trừ thành phần trơn, khác với gradient nhị phân dùng phát hiện biên. Vì vậy bản đồ Canny không được cộng trực tiếp thành đường trắng lên ảnh.

Canny tìm thay đổi cường độ, không phân biệt biên xe với vạch đường, cây, biển báo hay công trình. Khi gate Apply gần toàn bộ ảnh, chính sách cũng chưa thể hiện tính chọn lọc ngữ nghĩa.

Tăng biên không khôi phục thông tin vốn không có trong các xe rất nhỏ; kết luận về xe nhỏ cần AP theo kích thước với GT tương ứng, không suy ra chỉ từ tên lớp motorcycle hoặc bicycle.

3.10. Pseudocode và hợp đồng dữ liệu

Bảng 25. Luồng chung có thể tái lập từ notebook 01–04

Bước	Đầu vào / thao tác	Đầu ra cần giữ
1. Chốt	Đọc configs, splits, seed và checkpoint khởi tạo	Hash, tập ID theo vai trò, chính sách
2. Đo	Đọc ảnh được phép; đo quality từ ảnh gốc ở thang512	Metadata brightness/contrast/noise
3. Định tuyến	Raw→Skip; v1/v3/v4→gate; v2→Apply	Quyết định và lý do; không dùng GT chọn gate
4. Biến đổi	Dùng toán tử đúng version ở hình học nguồn	Ảnh uint8 ba kênh, cùng kích thước
5. Xuất	JPEG95 và nhãn YOLO đồng nhất hình học	data.yaml, metadata, nhãn, manifest
6. Train	Khởi tạo COCO độc lập; train S20, chọn trên calibration	args, results, best/last, marker hoàn tất
7. Eval	best.pt và development đúng chính sách	Overall, class, cohort, speed; giữ final kín

Không đổi nhãn khi ảnh nhìn rõ hơn. Mỗi hộp YOLO gồm class_id và tọa độ chuẩn hóa: x_center=(x1+x2)/(2W), y_center=(y1+y2)/(2H), width=(x2−x1)/W, height=(y2−y1)/H. Khi preprocessing không thay hình học, nhãn phải giữ nguyên. Resize/letterbox của dataloader là giai đoạn khác và chịu trách nhiệm ánh xạ hộp theo luồng native.

Trong data.yaml, test trỏ development để sử dụng API đánh giá của thư viện. Tập final_test là vai trò nghiên cứu khác và không được suy ra từ tên test của một file cấu hình. Tổng số ảnh được xuất cho một policy S20 là 9.781+6.985+13.975=30.741; đây là mẫu kiểm tra cơ bản để phát hiện thiếu hoặc lẫn vai trò khi chuẩn bị dữ liệu.

3.11. Chia nhóm và giới hạn bảo vệ rò rỉ

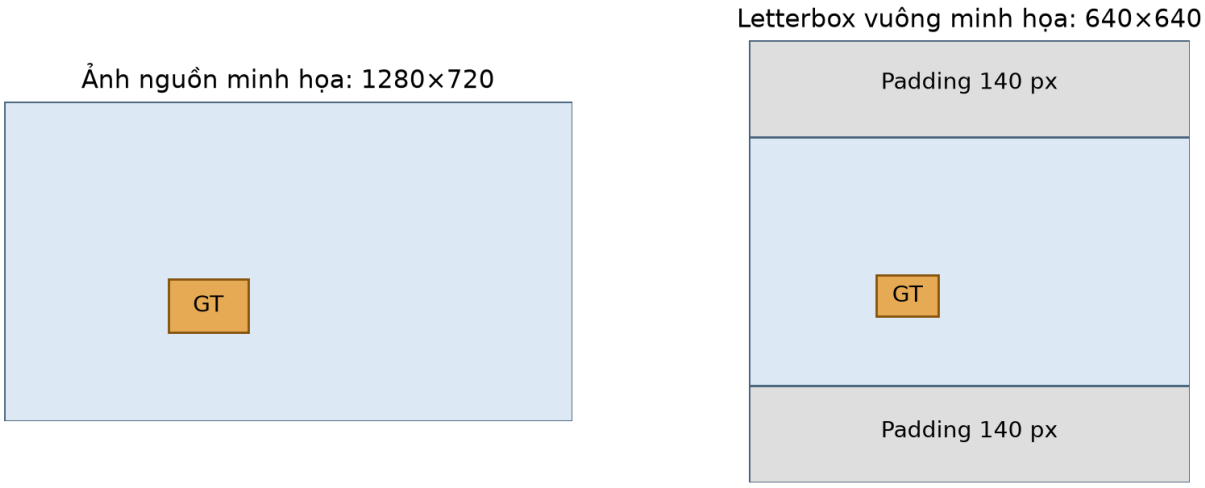
Issue5 ghi nhận không có video ID đáng tin cậy trong bản annotation đang dùng; nhóm nguồn được suy từ tiền tố tên BDD. Trong official train, các nhóm có ảnh trùng byte hoặc cùng dHash64 được hợp nhất bắc cầu trước phân vai trò. Đây là biện pháp bảo vệ theo nhóm có thể kiểm tra, không phải chứng minh đã loại được toàn bộ ảnh gần trùng hoặc mọi liên hệ video chưa biết.

Theo docs/ISSUE5_RESULTS.md của nhánh feat/issue-5-data-protocol, từ 10.000 official-val có 96 ảnh đã phơi lộ trước đây, 169 ảnh cùng nhóm với các ảnh đó và 5 ảnh cùng nhóm với official-train bị loại; còn 9.730 ảnh được dành làm final. Đây là thông tin từ hồ sơ audit, không phải kết quả mở lại ảnh final trong lần viết báo cáo. 69.863 ảnh official-train là quần thể phát hiện được trong bản nguồn dự án; chưa có chứng cứ cho phép kết luận toàn bộ chênh lệch so với số danh nghĩa của dataset là ảnh lỗi.

Hồ sơ Issue5 ở thời điểm đầu còn tính một phạm vi nhãn phương tiện khác, trong khi evaluator năm lớp hiện tại ghi development có 152.774 instance và 121 ảnh nền. Báo cáo ưu tiên số instance của output evaluator hiện tại cho các bảng AP; không gộp số nhãn lịch sử khác phạm vi vào cùng một mẫu số. Nhãn train đường sắt bị loại cũng cần được phân biệt với tên split train.

3.12. Từ ảnh nguồn tới tensor của YOLO26n

Kích thước nguồn và imgsz là hai khái niệm khác nhau. Policy chạy trước trên ảnh nguồn, giữ hình học và mã hóa JPEG95. Dataloader sau đó đọc JPEG và thực hiện biến đổi hình học theo chế độ train/eval. Luồng native đưa ảnh màu về thứ tự RGB và tensor B×3×H×W; cường độ được chia255 để về thang[0,1]. B là số ảnh trong batch. Bounding box được biến đổi tương ứng ở bước resize/letterbox; các giá trị H/W là kích thước tensor sau padding, không nhất thiết bằng kích thước JPEG nguồn.



Ví dụ hình học, không phải ảnh hay prediction mới. Eval rect=True có thể tạo tensor chữ nhật.

Hình 10. Ví dụ hình học khi letterbox vuông: ảnh1280×720 thu về640×360 rồi pad140pixel mỗi phía trên/dưới. Đây là minh họa; rect=True có thể dùng canvas chữ nhật thay vì640×640.

$$r = \min(H_{target} / H_{source}, W_{target} / W_{source})$$

$$x' = r \cdot x + pad_x; \quad y' = r \cdot y + pad_y$$

Trong ví dụ1280×720→640×640, r=0,5. Một hộp (370,200,550,320) trở thành (185,240,275,300) nếu dùng padding dọc140. Chiều rộng và chiều cao hộp cũng nhân0,5. Ví dụ này cho thấy vì sao không được chỉ resize ảnh mà giữ nguyên tọa độ pixel của nhãn. Nhãn YOLO chuẩn hóa của source và các hộp tensor được dataloader xử lý là hai giai đoạn liên tiếp.

Train rect=False và eval rect=True không mâu thuẫn: train có augmentation như mosaic, còn eval dùng luồng xác định để giữ đánh giá nhất quán. Batch chữ nhật được pad theo cơ chế native và stride, nên imgsz640 không bảo đảm mọi tensor eval là640×640. Các bảng feature map ở chương2 là minh họa cho input vuông; không được dùng để suy ngược kích thước batch từ mỗi dòng metric.

3.13. Mục tiêu học, cập nhật trọng số và chọn model

3.13.1. Ba thành phần loss quan sát trong project

results.csv ghi train/box_loss, train/cls_loss và train/l1_loss cùng các cột validation tương ứng. Mã BboxLoss của package đang cài dùng thành phần IoU cho độ khớp hộp; v8DetectionLoss dùng BCEWithLogits cho điểm lớp. reg_max=1 khiến nhánh regression này dùng L1 thay vì Distribution Focal Loss nhiều bin. Vì vậy không mô tả đồ án đang tối ưu DFL chỉ vì nhiều tài liệu YOLO cũ dùng tên loss đó [13].

$$L_{branch} = \lambda_{box} \cdot L_{box} + \lambda_{cls} \cdot L_{cls} + \lambda_{reg} \cdot L_{L1}$$

Bảng 26. Loss và ý nghĩa khi đọc đường cong

Thành phần	Cơ chế trong mã package	Diễn giải / giới hạn
box_loss	CIoU có trọng số cho foreground	Đo độ khớp hình học hộp trong mục tiêu học
cls_loss	BCEWithLogits với target score	Đo sai lệch điểm lớp so với nhãn được gán
l1_loss	L1 khoảng cách hộp chuẩn hóa khi reg_max=1	Bổ sung tín hiệu hồi quy, không là DFL đa bin
Trọng số B-Raw	box=7,5; cls=0,5; dfl=1,5 trong args	Tên dfl là gain phần regression trong implementation này

Công thức trên mô tả một nhánh. E2ELoss kết hợp loss one-to-many và one-to-one, với trọng số chuyển dần trọng tâm về nhánh one-to-one theo lịch của implementation. Các loss item được ghi

log xuất phát từ nhánh one-to-one theo mã cục bộ, không nhất thiết bằng tổng mục tiêu scalar dùng backward. Đây là lý do chỉ diễn giải các đường loss như tín hiệu quá trình học, không cộng tùy ý các giá trị log để khôi phục toàn bộ objective.

Loss giảm cho thấy model tối ưu tốt hơn mục tiêu học trên dữ liệu tương ứng, nhưng AP còn phụ thuộc xếp hạng confidence, ngưỡng IoU và tập đánh giá. Hai model có loss khác nhau vẫn có thể có thứ hạng AP ngược nhau. Khi chọn model, dự án dùng native fitness trên calibration; development dành để báo cáo sau chọn checkpoint.

3.13.2. Một vòng huấn luyện trong thí nghiệm

Bảng 27. Diễn giải chu kỳ train của từng policy

Pha	Hoạt động	Trạng thái cập nhật
Nạp batch	Đọc ảnh policy đã xuất; augmentation theo cấu hình	Không chọn policy mới từ GT/development
Forward	Backbone→neck→head; dự đoán đặc trưng/hộp/lớp	Tính toán với trọng số hiện tại
Gán target và loss	Ghép giám sát theo cơ chế native, tính loss	Có sử dụng nhãn S20 để tạo gradient
Backward / step	AMP theo recipe; SGD với momentum và decay	Cập nhật trọng số của model được train
Validation	Đánh giá calibration sau epoch theo native trainer	Không thực hiện optimizer step trên calibration
Lưu model	best.pt theo fitness; last.pt để tiếp tục khi phù hợp	Không chọn epoch bằng development

Một epoch là một lượt duyệt dữ liệu train theo cách dataloader tổ chức. Batch32 và batch16 tạo số mini-batch khác nhau trên S20, dù cùng50epoch. nbs=64 điều khiển quy ước tích lũy/chuẩn hóa trong trainer, không phải số ảnh vật lý luôn nằm trên GPU. AMP giúp thực thi train với độ chính xác hỗn hợp; đánh giá được ghi FP32 là lựa chọn riêng của evaluator.

Recipe đặt lr0=0,01, lrf=0,01, momentum=0,937, weight_decay=0,0005; warmup_epochs=3, warmup_momentum=0,8 và warmup_bias_lr=0,1. cos_lr=False và patience=0 được giữ theo cấu hình. Đây là thông số đồ án sử dụng; việc tài liệu dòng YOLO26 có trình bày MuSGD không có nghĩa các lượt train này đã dùng optimizer đó. Tên optimizer trong args.yaml là SGD.

3.13.3. Augmentation và lý do tắt thay đổi màu

Mosaic=1, fliplr=0,5, scale=0,5 và translate=0,1 cung cấp biến thiên hình học trong giai đoạn train; close_mosaic=10 chỉ định giai đoạn cuối đóng mosaic theo trainer. Các phép này khác preprocessing offline: được áp dụng trong dataloader khi train và không tạo một policy mới lưu cố định cho mọi ảnh. Cấu hình HSV đều bằng0 để tránh thay màu/độ sáng ngẫu nhiên làm khó diễn giải tác động Gamma, Local-shadow, CLAHE hoặc Canny.

Đánh giá dùng augment=False, cùng conf0,001, max_det300 và cùng định nghĩa GT trong mỗi nhóm. Không bật thêm test-time augmentation riêng cho một policy rồi quy toàn bộ lợi ích cho preprocessing. Tương tự, không chọn ảnh minh họa từ development dựa trên việc một policy trông đẹp hoặc dự đoán tốt hơn; các hình pixel của báo cáo lấy từ calibration như đã mô tả.

3.14. Đại lượng cần ước lượng và ma trận đối chứng

Đối chiếu chính trong các nhóm đã chạy là hiệu ứng chung của pipeline: model được train trên policy nào sẽ đọc ảnh theo policy đó khi eval. Ký hiệu f_q là checkpoint train trên policy q, T_q là toán tử ảnh cùng policy và M là metric trên cùng development. Hiệu ứng được trình bày theo điểm phần trăm, không lấy điểm calibration làm biến đầu ra thay thế.

$$\Delta_joint(q) = 100\times[M(f_q, T_q(D_dev)) - M(f_raw, T_raw(D_dev))]$$

Bảng 28. Các cặp so sánh được phép trong snapshot đang báo cáo

Nhóm	Cặp policy	Điều giữ chung	Phạm vi kết luận
A	v1/v2/v3 so với v0	Batch32, full50, seed2026, cùng ID/GT	Delta trong nhóm A
B	v1/v4 so với v0	Batch16, full50, seed2026, cùng ID/GT	Delta trong nhóm B
Giữa A/B	Cùng policy v0 hoặc v1	Nhiệm vụ/ID giống; runtime và batch khác	Mô tả độ ổn định; không cô lập nguyên nhân
A-v3 với B-v4	Không dùng như đối chứng trực tiếp	Có nhiều yếu tố thay đổi đồng thời	Không kết luận ưu thế nhân quả của policy

Seed2026 và checkpoint khởi tạo chung giúp kiểm soát thiết kế nhưng không biến kết quả thành định luật xác định độc lập nền tảng. Batch, worker, PyTorch/CUDA, trạng thái số học và lịch sử chọn checkpoint vẫn phải được ghi nhận. Việc A và B khác điều kiện là lý do tách bảng, không phải lý do bỏ kết quả v2/v3 đã lưu trong notebook.

3.15. Phân rã ảnh hưởng train và ảnh hưởng input: thiết kế mở rộng

Thiết kế sau là kế hoạch kiểm chứng, không được ghi như bốn kết quả mới của full50. Với mỗi policy q, cần hai detector đã train độc lập và hai cách tạo input trên cùng ID. Giữ nguyên từng checkpoint khi đổi input cho phép kiểm tra thay đổi pixel riêng biệt với thay đổi trọng số.

Bảng 29. Ma trận factorial 2×2 được đề xuất cho mỗi policy

Detector / input	Raw input	Processed input
Train Raw	A: f_raw trên T_raw(D)	B: f_raw trên T_q(D)
Train policy q	C: f_q trên T_raw(D)	D: f_q trên T_q(D)

$$\text{Hiệu ứng input trên model Raw} = B - A; \quad \text{joint pipeline} = D - A$$

$$\text{Tương tác} = (D - C) - (B - A)$$

Các biểu thức ở đây dùng metric trên cùng thang, nhân 100 nếu đổi từ fraction sang pp. Một ảnh Skip không bảo đảm detector train policy cho cùng output với detector Raw; vì vậy chỉ nhìn cohort Skip không cô lập ảnh hưởng model. Nếu ảnh processed đã được dọn sau export, phải tái xuất đúng config/manifest trước khi chạy bốn ô. Không coi sự tồn tại hàm factorial_bootstrap trong code là bằng chứng bốn ô đã được thực hiện.

3.16. Kiểm chứng tính nhất quán và kế hoạch độ bất định

Bảng 30. Các kiểm tra để chấp nhận một bảng kết quả

Lớp kiểm tra	Điều phải đúng	Nguồn đối chiếu
Membership	Đúng tập ID/role, không lẫn final	splits.csv.gz, data inventories
GT / hình học	Đúng năm lớp, kích thước và hộp nguồn	annotation_profile, verify_label
Policy	Đúng gate/operator, JPEG95, runtime	YAML, config snapshot, mã/hash
Train	Đúng recipe, độc lập khởi tạo, hoàn tất	args.yaml, results.csv, complete.json
Evaluation	Đúng best.pt/data/settings và role	metrics.json.inputs và hash
Tổng hợp	Mean AP lớp khớp overall trong sai số nguồn	Bảng class/overall, notebook output
Nguồn A	Giữ nguyên hash notebook, trích đúng cell/output	notebook_evidence.json

Các kiểm tra bảng/nguồn đã được dùng khi tạo báo cáo. Chúng xác minh tính nhất quán và danh tính chứng cứ, không thay cho chạy lặp nghiên cứu. Với nhóm A chỉ có output notebook, không suy rằng hash checkpoint gốc đã được kiểm tra trên máy hiện tại. Với nhóm B có artifact native, mức kiểm tra bao gồm marker, metric và checkpoint như đã mô tả.

Nếu phân tích khoảng tin cậy ở bước tiếp theo, paired bootstrap cần cùng ID và GT trước/sau, lấy mẫu lại cả nhóm nguồn với hoàn lại, tính lại AP rồi lấy phân vị2,5%/97,5%. Nhóm bị lặp trong một bootstrap draw phải giữ đầy đủ thành viên của nó. Những draw thiếu một lớp cố định không được âm thầm bù AP=0 vào macro; cần báo valid_draws và số lần resample.

Khoảng tin cậy như vậy có điều kiện trên các checkpoint đã train, không phản ánh biến thiên giữa seed, ngân sách tìm policy hay checkpoint selection. Để ước lượng biến thiên train cần lặp nhiều seed. Báo cáo full50 hiện không công bố một CI mới; khoảng tin cậy pilot960 được nêu riêng ở chương4 và không được chuyển sang nhóm A/B.

3.17. Đo chi phí và giới hạn thực nghiệm

Do preprocessing chạy ngoại tuyến, có ít nhất ba loại chi phí cần tách: chuẩn bị ảnh, huấn luyện và suy luận. Thời gian xuất gồm đọc nguồn, quality/gate, toán tử, JPEG và hash; native train/eval có luồng dữ liệu riêng. V4 dù Skip vẫn tính Canny để quyết định gate, nên chi phí Skip không được coi bằng 0 hoặc bằng Raw chỉ từ tên tuyến.

$$T_{end\text{-}to\text{-}end} = T_{đọc/giải_mã} + T_{policy} + T_{detector} + T_{hậu_xử_lý} + T_{I/O_khác}$$

Công thức là khung đo đề xuất; báo cáo chưa có phép đo tất cả hạng mục trên cùng phiên chạy triển khai. Không cộng máy móc các số ms/ảnh lấy từ nhiều máy hoặc các giai đoạn có batch khác nhau. speed_ms của evaluator không bao gồm toàn bộ policy ngoại tuyến; results.csv.time mô tả lượt train, không phải độ trễ camera.

Các hạn chế phương pháp còn gồm nhóm nguồn suy từ tên ảnh, một seed cho mỗi run, thiếu checkpoint gốc của nhóm A, hỗ trợ ít ở một số cohort/lớp và các yếu tố runtime khác giữa A/B. Phân tích sâu hơn sau khi thấy kết quả chỉ là mô tả, trừ khi có một kế hoạch mới và quần thể xác nhận thích hợp. Final giữ kín trong phạm vi báo cáo hiện tại; không có độ chính xác test cuối được suy ra từ development.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

4.1. Bản đồ chứng cứ và nguyên tắc đọc kết quả

Kết quả v2 và v3 có sẵn trong output đã lưu của notebook 04_eval.ipynb; không phụ thuộc vào việc thư mục runs gốc trên Linux có được tải về hay không. Báo cáo này trích đầy đủ bảng tổng thể, từng lớp, cohort và thông tin huấn luyện của v0–v3 từ các output đó. Đồng thời, số liệu mới của v4 được đối chiếu với các lần tái chạy Raw/Gamma cục bộ đúng nhóm B.

Snapshot nhóm B được chốt lúc 2026-11-12 15:39:25 (UTC+7). Nhóm A được xác định bằng run_date=2026-11-09 trong notebook, kèm hash từng file. Dấu gạch ngang có nghĩa chưa có chứng cứ phù hợp cho ô đó; không biểu thị AP bằng không. Không có kết quả nào trong chương này được lấy từ tập final holdout.

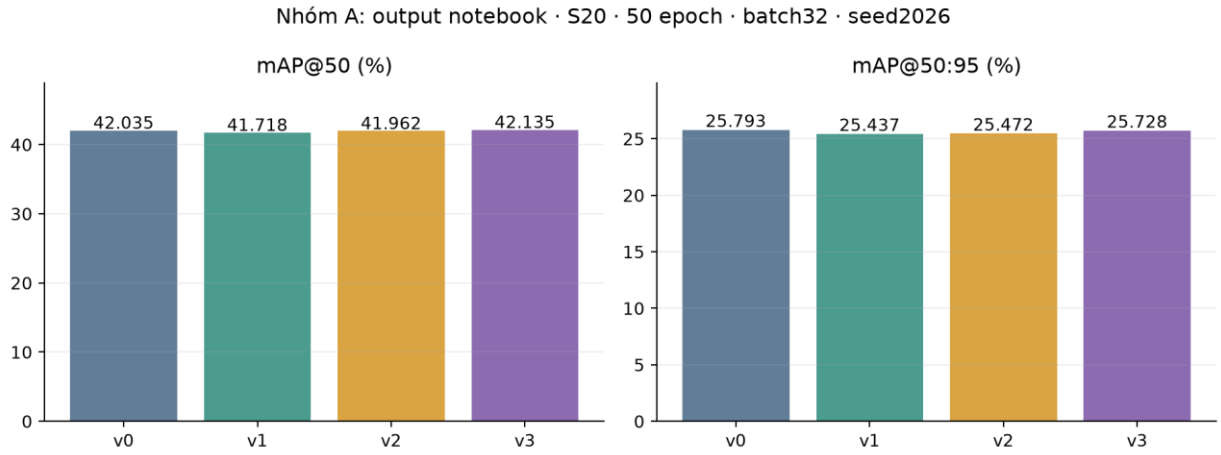
Bảng 31. Kiểm kê kết quả v0–v4 trong project

Policy	Nhóm A: full50/batch32	Nhóm B: full50/batch16	Nguồn chính
v0 Raw	Có output đầy đủ	Hoàn tất	A: v0/04_eval; B: v5_raw
v1 Gamma	Có output đầy đủ	Hoàn tất	A: v1/04_eval; B: v5_v1
v2 Local-shadow	Có output đầy đủ	Chưa có metric cuối khi chốt	A: v2/04_eval; B: v5_v2
v3 CLAHE	Có output đầy đủ	Chưa có metric cuối khi chốt	A: v3/04_eval; B: v5_v3
v4 Canny	Không thuộc nhóm A	Hoàn tất	B: v4-2026-11-10

4.2. Nhóm A: kết quả tổng thể v0–v3

Bảng 32. Nhóm A — development 13.975 ảnh, 152.774 GT; đơn vị %

Version	Precision	Recall	mAP@50	mAP@50:95
v0	58,0768	38,7199	42,0349	25,7931
v1	55,7908	39,3709	41,7177	25,4374
v2	57,8975	38,9667	41,9615	25,4724
v3	56,9054	38,9623	42,1348	25,7283



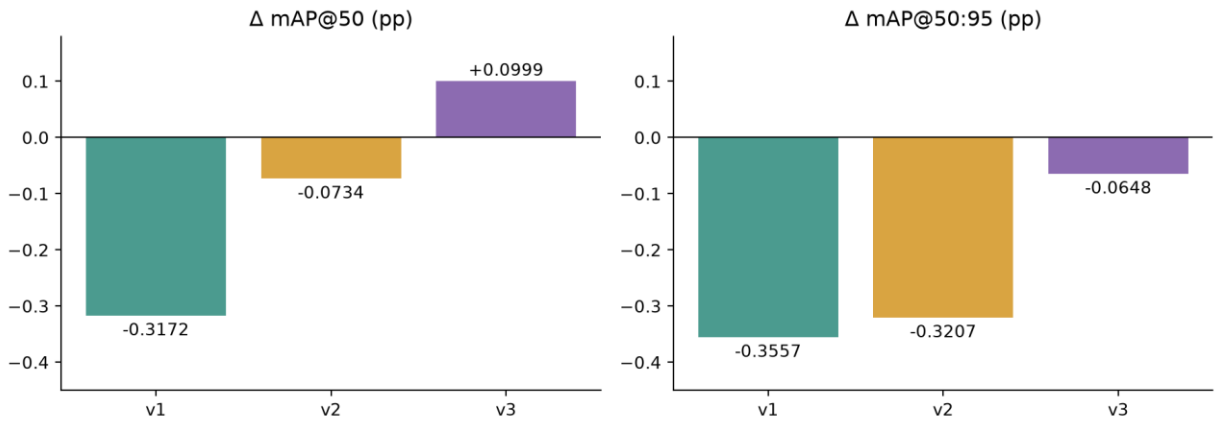
Hình 11. Nhóm A: tổng hợp số liệu từ output notebook 04_eval, cùng batch32, 50 epoch và seed2026.

Bảng 33. Nhóm A — chênh lệch từng policy so với A-v0, đơn vị pp

Version	**Δ Precision**	**Δ Recall**	**Δ mAP@50**	**Δ mAP@50:95**
v1	-2,2860	+0,6510	-0,3172	-0,3557
v2	-0,1793	+0,2468	-0,0734	-0,3207
v3	-1,1714	+0,2424	+0,0999	-0,0648

Trong nhóm A, v3 có mAP@50 cao nhất: 42,1348%, so với Raw 42,0349%, chênh +0,0999 pp. Tuy nhiên mAP@50:95 của v3 là 25,7283%, thấp hơn Raw 25,7931% khoảng 0,0648 pp. Vì thước đo chính của dự án là mAP@50:95, dữ liệu không xác nhận v3 vượt Raw toàn diện. Ưu thế ở IoU=0,50 không tự bảo đảm ưu thế khi yêu cầu định vị hộp chặt hơn.

V2 đạt mAP@50 41,9615% và mAP@50:95 25,4724%, thấp hơn Raw tương ứng 0,0734 và 0,3207 pp. V1 đạt 41,7177% và 25,4374%, cũng thấp hơn Raw trong nhóm này. V2 cao hơn v1 0,2438 pp về mAP@50 và 0,0350 pp về mAP@50:95, nhưng việc vượt một policy tiền xử lý khác không thay thế yêu cầu vượt đối chứng Raw.



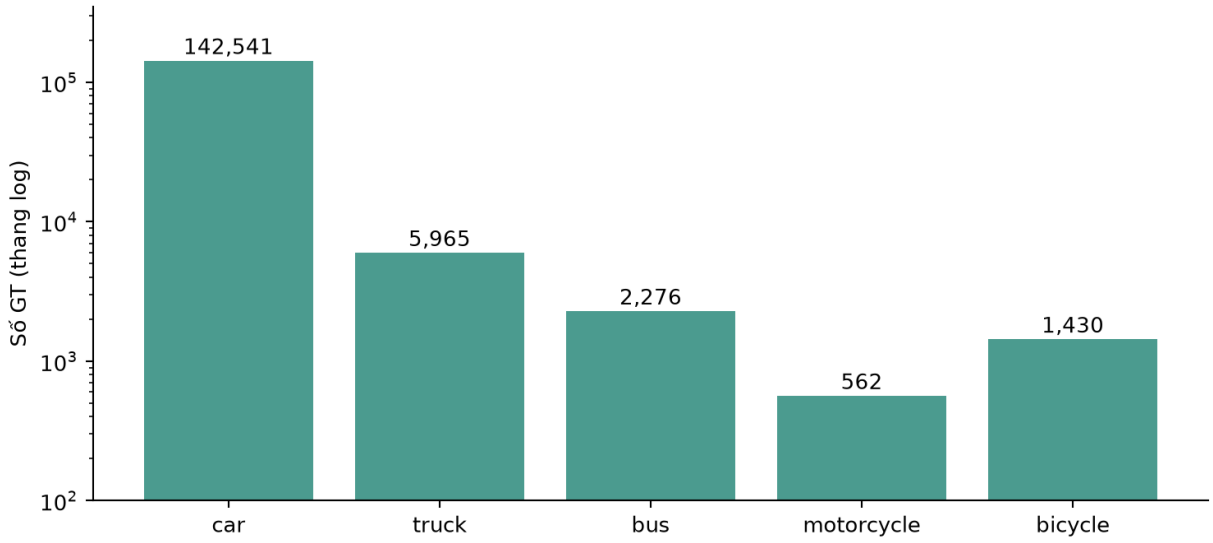
Hình 12. Nhóm A: delta được tính bằng 100×(metric_policy−metric_Raw); đường 0 là đối chứng.

Sắp hạng mô tả ở mAP@50 là v3 > v0 > v2 > v1, còn ở mAP@50:95 là v0 > v3 > v2 > v1. Đây là thứ tự của một lần chạy seed2026, chưa có sai số giữa seed hoặc kiểm định paired bootstrap cho full50. Chênh lệch nhỏ cần được mô tả đúng độ lớn, tránh dùng từ cải thiện rõ rệt hay có ý nghĩa thống kê khi chưa có kiểm định.

4.3. Nhóm A: AP theo lớp và mất cân bằng hỗ trợ

Bảng 34. Hỗ trợ GT development của năm lớp

Class	Ảnh có lớp	GT instance	Tỷ trọng GT (%)
car	13.820	142.541	93,302
truck	3.773	5.965	3,904
bus	1.759	2.276	1,490
motorcycle	435	562	0,368
bicycle	872	1.430	0,936



Hình 13. Phân bố GT development trên thang log; một ảnh có thể chứa nhiều lớp nên số ảnh theo lớp không cộng thành tổng ảnh.

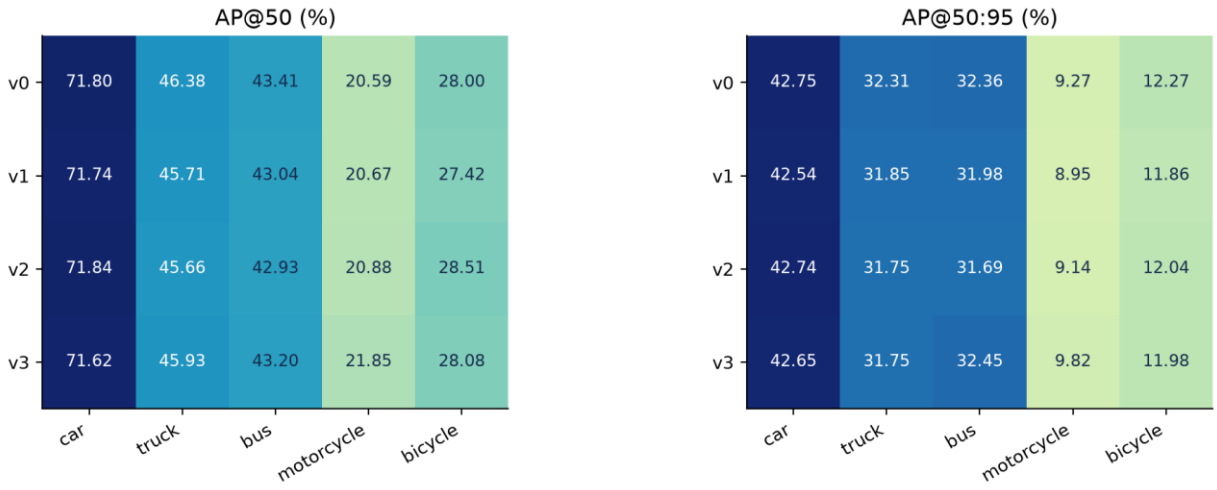
Car chiếm phần lớn instance, trong khi motorcycle chỉ có 562 và bicycle có 1.430 GT. Tuy vậy mAP native ở đây lấy trung bình AP theo năm lớp, không lấy trọng số theo số instance. Hai lớp xe hai bánh do đó vẫn có ảnh hưởng đáng kể lên mAP. Số hỗ trợ này mô tả development; không dùng nó để khẳng định tỷ lệ nhãn của S20 hoặc toàn bộ BDD100K.

Bảng 35. Nhóm A — AP@50 theo lớp, đơn vị %

Class	**v0**	**v1**	**v2**	**v3**
car	71,8038	71,7444	71,8352	71,6163
truck	46,3800	45,7116	45,6566	45,9348
bus	43,4062	43,0409	42,9259	43,1985
motorcycle	20,5864	20,6733	20,8801	21,8471
bicycle	27,9984	27,4182	28,5099	28,0773

Bảng 36. Nhóm A — AP@50:95 theo lớp, đơn vị %

Class	**v0**	**v1**	**v2**	**v3**
car	42,7507	42,5435	42,7384	42,6487
truck	32,3140	31,8501	31,7533	31,7497
bus	32,3641	31,9843	31,6910	32,4479
motorcycle	9,2685	8,9514	9,1421	9,8168
bicycle	12,2682	11,8577	12,0375	11,9783



Hình 14. Nhóm A: AP@50 và AP@50:95 của từng lớp từ các bảng output đã lưu; hai biểu đồ dùng thang màu khác nhau.

4.3.1. V2: kết quả chi tiết Local-shadow

Bảng 37. A-v2 so với A-v0 — từng lớp, đơn vị pp

Class	**Δ AP@50**	**Δ AP@75**	**Δ AP@50:95**
car	+0,0314	+0,0500	-0,0123
truck	-0,7234	-0,8804	-0,5607
bus	-0,4803	-0,9618	-0,6731
motorcycle	+0,2937	+0,0698	-0,1264
bicycle	+0,5115	-0,1643	-0,2307

V2 tăng AP@50 của car rất nhỏ (+0,0314 pp), motorcycle (+0,2937 pp) và bicycle (+0,5115 pp), nhưng giảm truck (−0,7234 pp) và bus (−0,4803 pp). Tổng macro mAP@50 vẫn giảm. Tại AP@50:95, cả năm lớp đều thấp hơn Raw; mức giảm bus và truck lớn hơn car. Đây là ví dụ cụ thể cho việc nâng sáng cục bộ có thể làm một số detection qua ngưỡng IoU0,50 mà chưa tạo lợi ích trên toàn dải IoU.

Precision v2 tổng thể gần Raw, recall tăng nhẹ, song không thể suy số TP/FP/FN tuyệt đối từ hai tỷ lệ này: evaluator chọn điểm vận hành native và còn thực hiện trung bình theo lớp. Báo cáo giữ riêng P/R, AP@50, AP@75 và AP@50:95 để người đọc thấy các khía cạnh khác nhau của kết quả; không gọi chúng là bốn phép đo độc lập có thể bỏ phiếu chọn phương pháp.

4.3.2. V3: kết quả chi tiết tăng tương phản thích nghi

Bảng 38. A-v3 so với A-v0 — từng lớp, đơn vị pp

Class	Δ AP@50	Δ AP@75	Δ AP@50:95
car	-0,1875	-0,0275	-0,1020
truck	-0,4452	-0,6652	-0,5643
bus	-0,2077	+0,0279	+0,0838
motorcycle	+1,2607	-0,0753	+0,5483
bicycle	+0,0789	-0,6348	-0,2899

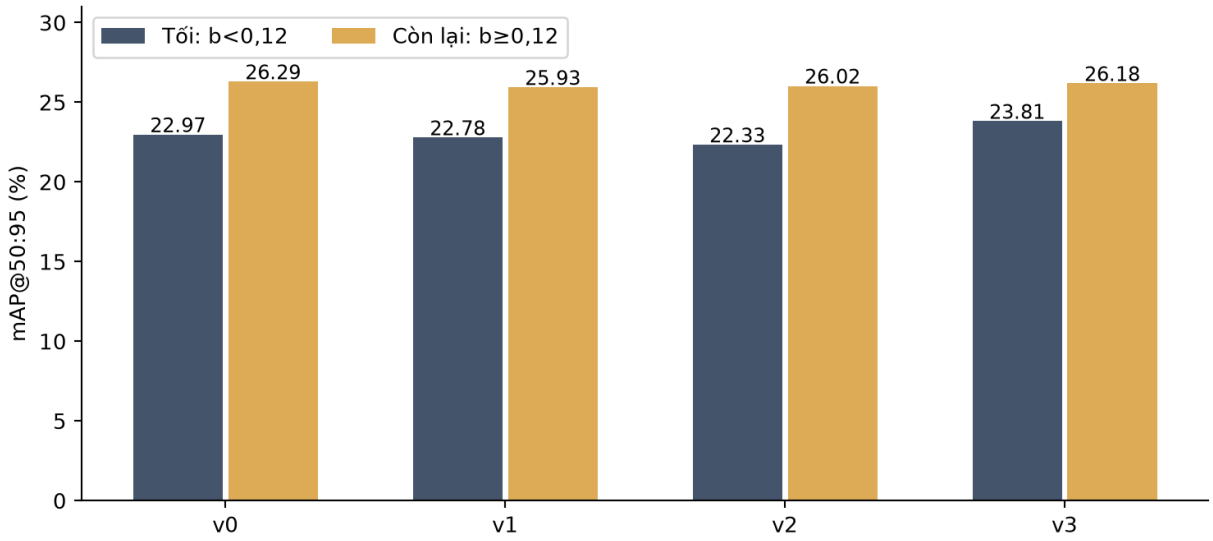
Đóng góp tăng AP@50 lớn nhất của v3 nằm ở motorcycle: 21,8471% so với Raw 20,5864%, tức +1,2607 pp. Car, truck và bus giảm nhẹ; bicycle tăng +0,0789 pp. Vì thế v3 đạt mAP@50 cao nhất không đồng nghĩa tốt nhất cho từng loại xe. Ở AP@50:95, motorcycle và bus có lợi ích, còn car, truck và bicycle giảm, khiến mức trung bình thấp hơn Raw.

Đặc biệt, AP@75 của motorcycle ở v3 là 5,9929%, thấp hơn Raw 6,0682%, dù AP@50 và AP@50:95 của lớp này tăng. AP@50:95 là trung bình trên mười ngưỡng nên một ngưỡng riêng không nhất thiết có cùng chiều biến động. Vì vậy không diễn giải tất cả cải thiện thành định vị tốt hơn ở mọi IoU.

4.4. Nhóm A: cohort tối/sáng và định tuyến

Bảng 39. Nhóm A — cohort cùng định nghĩa theo ảnh gốc, mAP@50:95 (%)

Cohort	Số ảnh	v0	v1	v2	v3
b<0,12	3.307	22,9681	22,7799	22,3322	23,8088
b≥0,12	10.668	26,2939	25,9344	26,0236	26,1809



Hình 15. Nhóm A: dark/bright là ngưỡng brightness, không đồng nghĩa trực tiếp với metadata night/day.

Trên 3.307 ảnh tối, v3 đạt 23,8088% so với Raw 22,9681%, tăng +0,8407 pp về mAP@50:95. Trên 10.668 ảnh còn lại, v3 đạt 26,1809% so với Raw 26,2939%, giảm −0,1130 pp. Đây là một tín hiệu tích cực có điều kiện ở nhóm tối, nhưng chưa đủ để khuyến nghị áp dụng cho toàn bộ dữ liệu. V2 giảm −0,6359 pp ở nhóm tối và −0,2703 pp ở nhóm còn lại; giả thuyết Local-shadow giúp nhóm tối chưa được xác nhận trong nhóm A.

Không lấy 3.307/13.975 nhân AP_dark cộng phần còn lại để tái tạo overall mAP. AP của tập hợp được tính từ đường precision–recall sau gộp detection, không phải trung bình tuyến tính AP của từng cohort theo số ảnh. Điều này giải thích vì sao mức tăng ở nhóm tối không bảo đảm overall tăng cùng tỷ lệ.

Bảng 40. Nhóm A — tỷ lệ Apply theo tập, đơn vị %

Policy	S20/train	Calibration/val	Development/test
v0	0,0000	0,0000	0,0000
v1	22,6766	22,4481	23,6637
v2	100,0000	100,0000	100,0000
v3	40,0368	40,0000	40,4508

V1 xử lý 23,6637% development và chỉ định tuyến theo brightness. V2 xử lý 100% ảnh. V3 xử lý 40,4508% development, tương ứng 5.653 ảnh Apply và 8.322 ảnh Skip. Sự khác nhau giữa tỷ lệ

train/calibration/development là thống kê của dữ liệu, không phải thay đổi ngưỡng giữa các tập. V3 Apply không trùng tập ảnh tối vì gate còn sử dụng contrast, noise và tỷ lệ cực trị.

Bảng 41. Nhóm A — cohort định tuyến đã lưu, mAP@50:95 (%)

Policy	Cohort	Ảnh	mAP@50:95
v1	apply	3.307	22,7799
v1	skip	10.668	25,9344
v3	apply	5.653	24,2665
v3	skip	8.322	26,4534

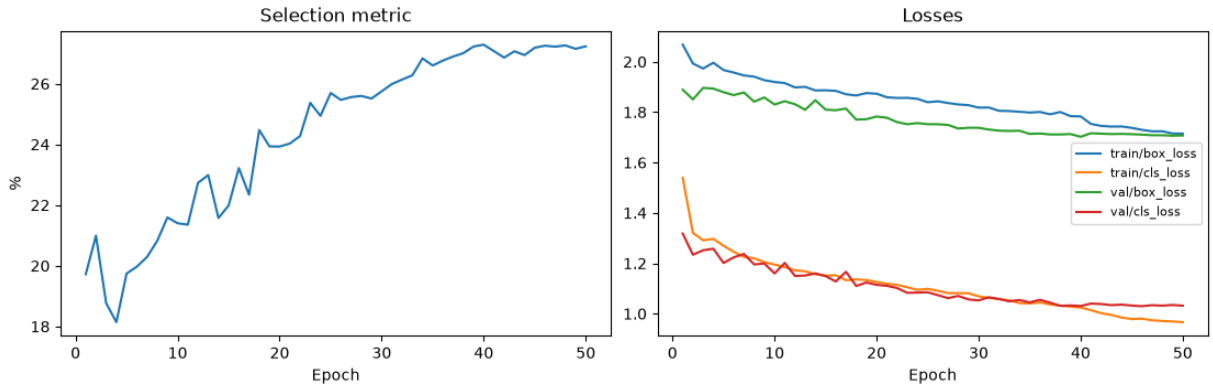
Apply/Skip là hai quần thể khác nhau về độ khó, số lớp có hỗ trợ và phân phối ảnh. Không dùng chênh lệch AP giữa chúng làm độ tăng do toán tử. Ngay cả ảnh Skip vẫn đi qua detector đã được fine-tune với tập train của policy đó; nên Skip ở v3 không bắt buộc có dự đoán trùng Raw. Đánh giá tác động pixel riêng cần cùng detector, cùng tập ID và chỉ đổi input.

4.5. Nhóm A: lựa chọn checkpoint, hội tụ và thời gian

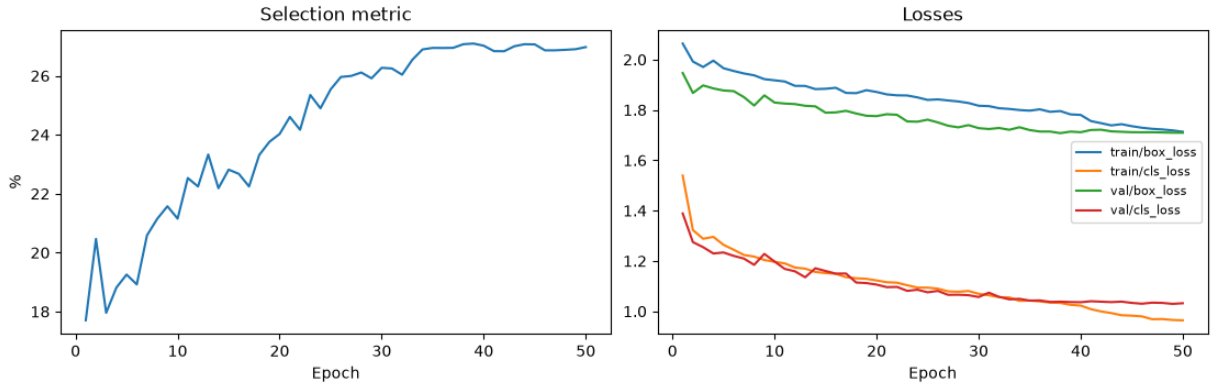
Bảng 42. Nhóm A — tóm tắt huấn luyện từ output 03_train

Version	Epoch	Epoch đỉnh calib	Calib mAP@50:95 (%)	Train giờ (log)
v0	50	40	27,28	0,878
v1	50	39	27,09	—
v2	50	40	27,21	0,891
v3	50	39	27,05	0,913

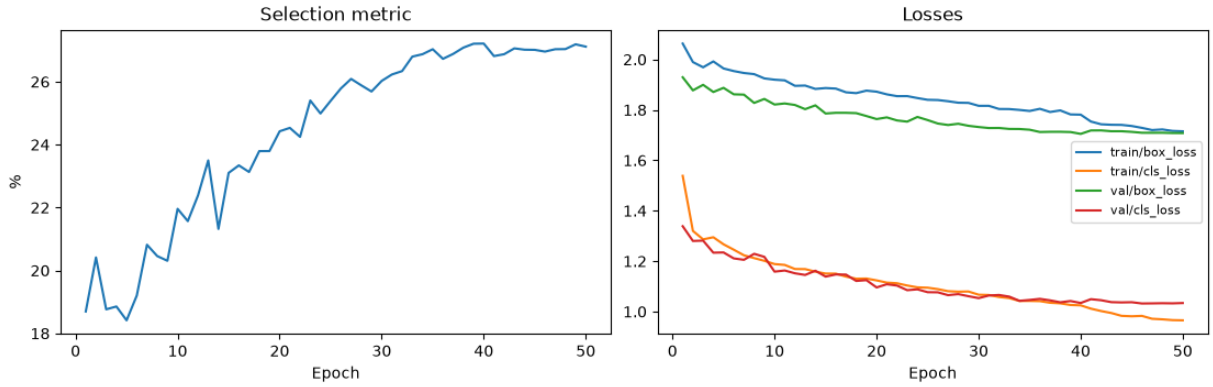
Tất cả bốn notebook ghi nhận 50 epoch. Giá trị calibration trong dòng summary được làm tròn bốn chữ số ở thang 0–1, vì vậy bảng không bổ sung độ chính xác giả. V1 không còn đủ stream log để xác định tổng giờ huấn luyện nên để trống. Các đỉnh nằm ở epoch39–40; đây là thông tin chọn checkpoint trên calibration, không phải kết quả development tại chính các epoch đó.



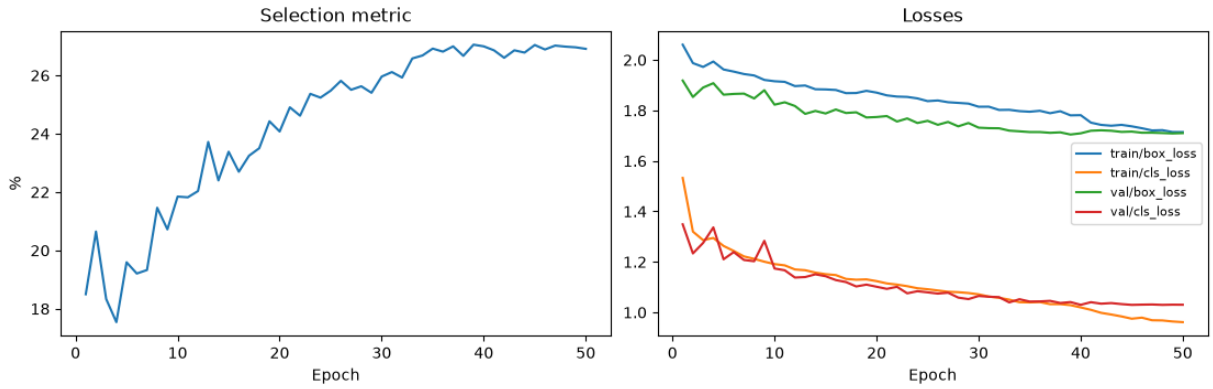
Hình 16. Nhóm A — v0: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có.



Hình 17. Nhóm A — v1: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có.



Hình 18. Nhóm A — v2: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có.



Hình 19. Nhóm A — v3: đường cong huấn luyện/calibration gốc nhúng trong 03_train.ipynb; giữ nguyên hình, không tái dựng dữ liệu chưa có.

Bảng 43. Nhóm A — tốc độ native evaluator, ms/ảnh

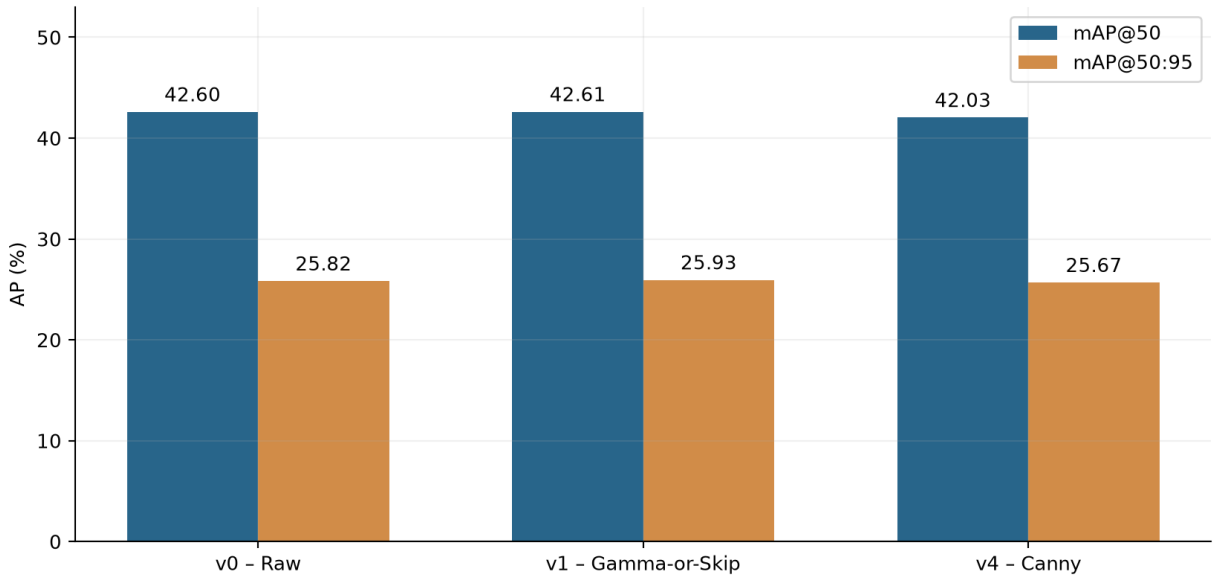
Version	Preprocess	Inference	Postprocess
v0	0,1941	1,3396	0,1992
v1	0,2105	1,3326	0,2091
v2	0,1951	1,1058	0,1104
v3	0,1850	1,0560	0,0979

Inference dao động từ khoảng 1,06 đến 1,34 ms/ảnh trong log nhóm A. Đây là thời gian native trên GPU máy gốc khi đọc ảnh đã preprocessing; không chứa chi phí chạy Gamma/Local-shadow/CLAHE ở giai đoạn xuất ảnh ngoại tuyến. Không thể dùng bảng này để kết luận v3 rẻ hơn Raw về toàn pipeline hoặc nhân nghịch đảo thành FPS hệ thống triển khai.

4.6. Nhóm B: đối chiếu v4 với Raw/Gamma cùng recipe

Bảng 44. Nhóm B — kết quả development đã hoàn tất, đơn vị %

Version	Precision	Recall	mAP@50	mAP@50:95
v0	56,2579	39,7743	42,5961	25,8215
v1	57,5943	39,4671	42,6109	25,9328
v4	56,7137	39,4288	42,0345	25,6708



Hình 20. Nhóm B: chỉ những lượt đã hoàn tất có điểm; các lượt v2/v3 của nhóm A đã được báo cáo riêng phía trên.

V4 đạt mAP@50 42,0345% và mAP@50:95 25,6708%, so với B-Raw 42,5961% và 25,8215%. Delta v4−Raw tương ứng -0,5616 pp và -0,1507 pp. Chính sách Canny hiện tại chưa cải thiện hai thước đo tổng thể trong điều kiện đã chạy.

B-v1 đạt mAP@50 42,6109% và mAP@50:95 25,9328%; tăng so với B-Raw lần lượt +0,0148 và +0,1113 pp. Mức tăng còn nhỏ và chỉ có một seed. Đây không phải cùng run với kết quả Gamma nhóm A.

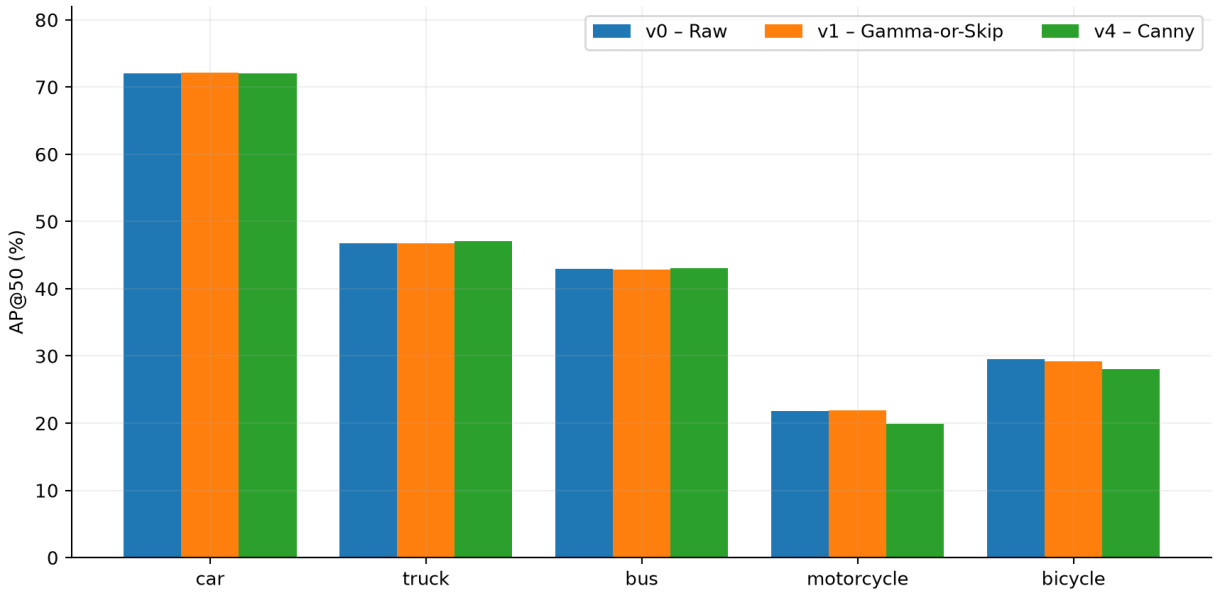
Bảng 45. Nhóm B — AP@50 từng lớp (%) và delta v4−Raw

Class	**v0**	**v1**	**v4**	**Δ v4−Raw (pp)**
car	72,0140	72,1455	72,0872	+0,0732
truck	46,7364	46,8128	47,0376	+0,3012
bus	42,9552	42,8993	43,0579	+0,1027
motorcycle	21,7510	21,9524	19,9016	-1,8494
bicycle	29,5237	29,2444	28,0882	-1,4355

Bảng 46. Nhóm B — AP@50:95 từng lớp (%) và delta v4−Raw

Class	**v0**	**v1**	**v4**	**Δ v4−Raw (pp)**
car	42,7695	42,7838	42,7682	-0,0013
truck	32,4448	32,5066	32,6579	+0,2131

Class	v0	v1	v4	Δ v4−Raw (pp)
bus	31,9535	31,9933	32,0282	+0,0747
motorcycle	9,3282	9,6448	8,6544	-0,6738
bicycle	12,6115	12,7356	12,2454	-0,3661

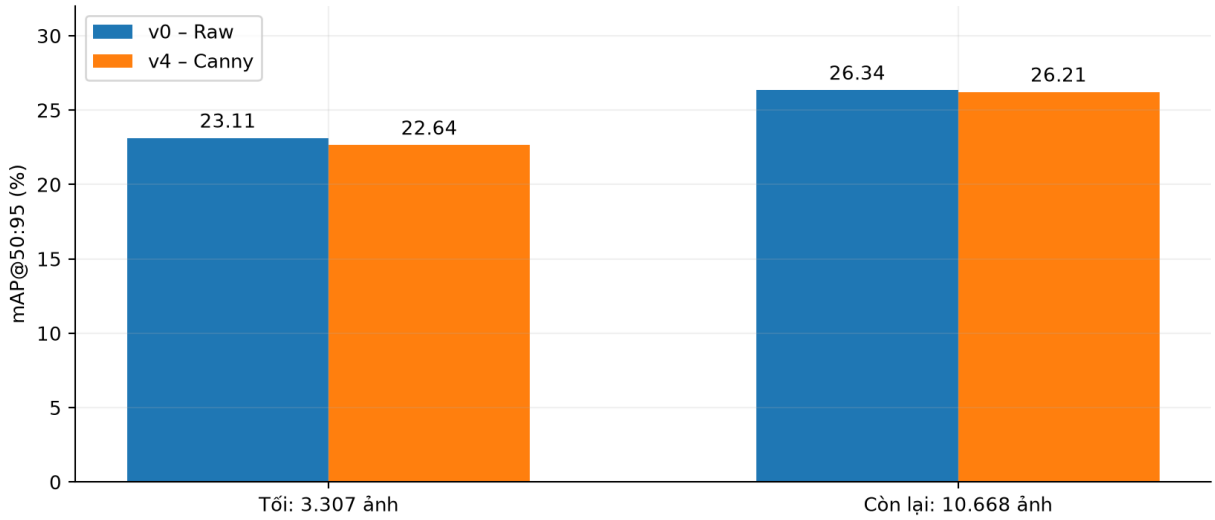


Hình 21. Nhóm B: AP@50 từng lớp; số GT development giống nhóm A.

Hai lớp motorcycle và bicycle là các điểm giảm đáng chú ý về AP@50 khi chuyển từ B-Raw sang v4. Điều này hỗ trợ việc kiểm tra riêng xe hai bánh trước khi triển khai. Chưa có bảng lỗi theo kích thước hay độ che khuất của nhóm B nên không khẳng định nguyên nhân là xe nhỏ, biên nền hoặc thiếu sáng; đó là các giả thuyết cần kiểm tra tiếp.

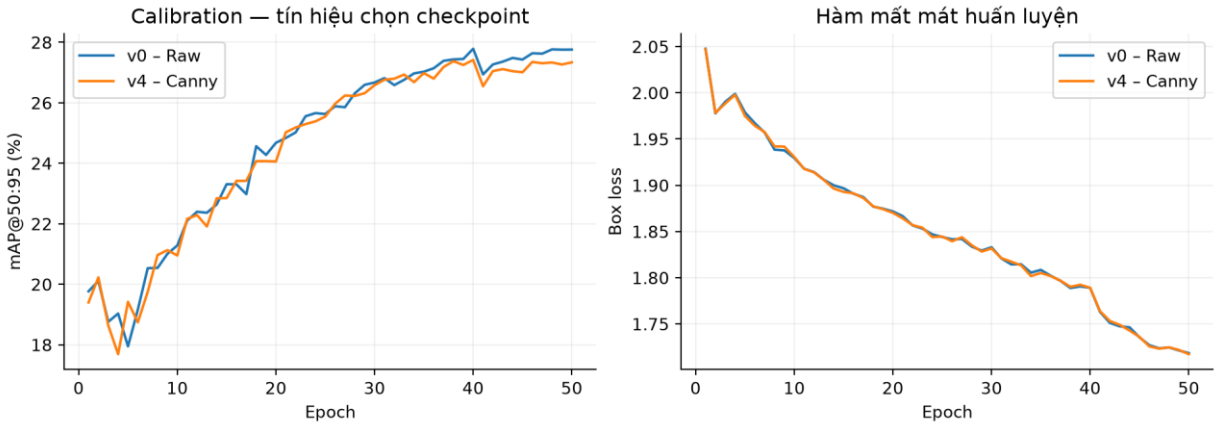
Bảng 47. Nhóm B — cohort cố định, mAP@50:95 (%)

Cohort	Ảnh	v0	v1	v4	Δ v4−Raw
dark	3.307	23,1136	23,4927	22,6386	-0,4750
bright	10.668	26,3354	26,4060	26,2087	-0,1267



Hình 22. Nhóm B: đối chiếu Raw/v4 trên cùng cohort theo brightness ảnh gốc.

V4 Apply 13.750/13.975 ảnh development (98,39%) và Skip 225 ảnh. Gate bỏ qua rất ít ảnh. Cohort Skip còn có thể vắng lớp bicycle; evaluator cohort chỉ trung bình các lớp có hỗ trợ, nên AP Skip không dùng xếp hạng trực tiếp với AP của toàn bộ năm lớp.



Hình 23. Nhóm B: đường cong Raw/v4 lấy từ results.csv native, không trộn với đường cong notebook nhóm A.

Bảng 48. Nhóm B — train và native inference của các lượt hoàn tất

Version	Train giờ	Epoch đỉnh calib	Inference ms/ảnh
v0	5,289	40	3,3013
v1	5,461	40	4,0280
v4	5,467	40	3,7508

4.7. Đối chiếu giữa nhóm A và B: độ ổn định của kết luận

Bảng 49. Cùng policy, khác nhóm thực thi; đơn vị %

Policy	A mAP@50	B mAP@50	A mAP@50:95	B mAP@50:95
v0	42,0349	42,5961	25,7931	25,8215
v1	41,7177	42,6109	25,4374	25,9328

Gamma thấp hơn Raw trong nhóm A nhưng nhỉnh hơn Raw trong nhóm B. Dữ liệu cho thấy kết luận của một lần chạy chưa chắc giữ nguyên khi thay môi trường/recipe. Vì batch, worker, GPU và phiên bản PyTorch thay đổi đồng thời, chưa thể quy nguyên nhân cho riêng batch hoặc phần cứng. Cũng không thể lấy chênh B-v4 với A-v3 làm bằng chứng trực tiếp Canny tốt hơn hay kém hơn CLAHE.

Để có bảng so sánh đủ năm policy trong cùng nhóm, cần đợi các lượt B-v2 và B-v3 hoàn tất, xác minh lifecycle và giữ recipe nhóm B. Kết quả A-v2/A-v3 vẫn là kết quả thực nghiệm hợp lệ có nguồn rõ ràng; việc B còn thiếu là thiếu lượt lặp trong nhóm mới, không phải thiếu mọi kết quả của hai phiên bản.

4.8. Kết quả lịch sử trong issue và phạm vi suy luận

Bảng 50. Lịch sử Issue45 — S20/640, 20 epoch, freeze11, AdamW FP32

Version	mAP@50 (%)	mAP@50:95 (%)	Nhóm
v0	35,1576	20,6124	H: khác full50
v1	35,2484	20,6879	H: khác full50
v2	35,3136	20,4930	H: khác full50

Issue45_S20_EXECUTION.md mô tả v2 được huấn luyện đủ lifecycle, với batch16, backbone freeze11, AdamW FP32 và 20 epoch. Calibration chọn checkpoint bằng ảnh Raw chung của v0. Trong điều kiện đó v2 tăng mAP@50 khoảng +0,1560 pp nhưng giảm mAP@50:95 khoảng −0,1194 pp so với Raw. Khi chuyển sang full fine-tune nhóm A, v2 lại giảm cả hai metric so với Raw. Hai nghiên cứu cho thấy khác biệt về recipe có thể đi cùng thay đổi kết quả; không được nhập số 20,49% vào bảng full50 25,47%.

Đối với v3, ISSUE47_V3_960_RESULTS.md còn ghi một pilot khác: detector Raw S20/960 đã huấn luyện được giữ nguyên, đánh giá trên 1.479 ảnh exploration, so sánh Raw/Always-on/Adaptive. mAP@50:95 lần lượt là 26,011103% / 26,034995% / 26,158582%. Adaptive tăng +0,147479 pp so với Raw, thấp hơn gate +0,2 pp; khoảng tin cậy 95% có điều kiện theo nhóm nguồn là [−0,050963; +0,415904] pp. Hồ sơ này kết luận dừng trước bước xác nhận/huấn luyện của nghiên cứu đó.

Pilot 960 là kiểm tra thay input trên detector cố định ở tập exploration, khác thí nghiệm A-v3 full50/640 với detector được train riêng và đánh giá development. Khoảng tin cậy của pilot không được dùng làm khoảng tin cậy cho số 42,1348% hoặc 25,7283% của A-v3. Quyết định không promote ở pilot cũng không có nghĩa notebook A-v3 không được chạy; mỗi kết luận phải gắn đúng nghiên cứu và mốc thời gian.

4.9. Trả lời câu hỏi nghiên cứu và giới hạn

Bảng 51. Tổng hợp trả lời RQ từ chứng cứ hiện có

Câu hỏi	Kết luận có căn cứ	Điều chưa được xác nhận
RQ1: tăng AP?	A-v3 tăng AP50 rất nhỏ; A-Raw cao nhất AP50:95. B-v4 giảm cả hai so với B-Raw.	Không có winner chung đủ năm policy cùng recipe; chưa có multi-seed.
RQ2: lợi ích ở nhóm/lớp?	A-v3 tăng AP50:95 nhóm tối; A-v2 giảm cả tối và còn lại; B-v4 có suy giảm xe hai bánh.	Chưa có phân rã đầy đủ theo kích thước, thời tiết và che khuất.
RQ3: chi phí và tái lập?	Có routing, log train và native speed; nguồn A/B được phân cấp.	Native speed không là end-to-end; checkpoint A chưa kiểm tra độc lập.

Các giá trị AP cao hay thấp chỉ phản ánh development đang dùng; tập final chưa mở nên chưa có ước lượng tổng quát hóa cuối cùng. Dataset nhóm nguồn được suy từ tên ảnh có hạn chế về bảo vệ video. Chỉ một seed và các chênh lệch cỡ phần mười điểm phần trăm làm cho kết luận về ưu thế ổn định còn yếu. Càng phân tích nhiều cohort sau khi đã xem kết quả, càng cần ghi rõ chúng là phân tích mô tả và tránh quay lại tối ưu ngưỡng trên development.

Báo cáo đã phân biệt ảnh minh họa, nhãn GT và dự đoán detector. Các hình preprocessing chỉ chứng minh biến đổi hoạt động đúng ý tưởng, không là chứng cứ tăng AP. Không có hình TP/FP/FN mới hoặc bảng AP-small/medium/large được tạo giả khi thiếu prediction artifact. Những phân tích đó được chuyển thành kế hoạch kiểm chứng có điều kiện ở chương sau.

4.10. Diễn giải kết quả theo cơ chế của detector

Bảng 52. Từ quan sát đến giả thuyết: tránh suy luận vượt dữ liệu

Quan sát đã đo	Giải thích được phép	Phép kiểm tra còn cần
A-v3 tăng AP50 nhưng giảm AP50:95	Ưu thế không đồng nhất giữa các ngưỡng định vị	Phân rã AP từng IoU, paired prediction và nhiều seed
A-v2 Apply100% và giảm metric tổng	Việc nâng sáng mọi ảnh chưa đủ tạo lợi ích detector	Cùng checkpoint so Raw/processed; đo thay đổi pixel và lỗi
B-v4 giảm AP xe hai bánh	Cần kiểm tra riêng hai lớp trong điều kiện đã chạy	AP theo cỡ hộp/che khuất; GT đủ hỗ trợ
Gamma đổi chiều delta giữa A/B	Kết luận một run nhạy với nhóm thực thi	Thiết kế giữ mọi yếu tố trừ một; không quy ngay cho GPU

YOLO26n nhận ảnh sau resize và tạo đặc trưng học được, vì vậy không phải mọi cạnh được Canny phát hiện đều là thông tin model cần. Một nét nền mạnh có thể đồng thời thay đổi đáp ứng feature map, trong khi biên của xe xa có thể đã yếu sau resize. Đây là một cơ chế khả dĩ cần kiểm tra, chưa phải nguyên nhân được chứng minh bởi bảng AP. Mục tiêu của báo cáo là kết nối thiết kế toán tử với câu hỏi thực nghiệm cụ thể, thay vì giải thích hậu nghiệm mọi tăng/giảm bằng một nguyên nhân duy nhất.

Nếu mục tiêu ứng dụng ưu tiên mAP@50, A-v3 là ứng viên đáng tái kiểm chứng vì tăng0,0999pp trong nhómA. Nếu ưu tiên độ khớp hộp trên nhiều IoU, A-Raw vẫn dẫn nhómA. Với nhómB đã hoàn tất, Gamma nhỉnh hơn Raw nhưng Canny chưa vượt đối chứng. Những lựa chọn này là ứng viên theo mục tiêu và nhóm thí nghiệm, chưa là quyết định triển khai cuối cùng cho toàn bộ BDD100K hoặc camera ngoài thực tế.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận từ năm phiên bản

Đồ án hệ thống hóa năm cách xử lý ảnh cho bài toán phát hiện car, truck, bus, motorcycle và bicycle trên BDD100K: Raw, Gamma-or-Skip, Local-shadow, tương phản cục bộ có gate và tăng chi tiết theo Canny. Các chính sách giữ nguyên hình học và nhãn, dùng JPEG95 và detector YOLO26n khởi tạo từ checkpoint COCO. S20 có 9.781 ảnh; calibration có 6.985 ảnh; development có 13.975 ảnh. Vai trò final được tách riêng và giữ kín.

Nhóm A đã có đầy đủ output v0–v3. V3 đạt mAP@50 cao nhất 42,1348%, hơn Raw 0,0999 pp, song mAP@50:95 25,7283% thấp hơn Raw 25,7931%. V2 đạt 41,9615% / 25,4724%, thấp hơn Raw ở cả hai metric tổng thể. Lợi ích đáng chú ý của v3 là mAP@50:95 nhóm ảnh tối tăng +0,8407 pp; điều này chưa đủ để khẳng định ưu thế chung hoặc độ tin cậy qua nhiều seed.

Trong nhóm B có đối chứng cùng batch16, v4 Canny đạt mAP@50 42,0345% và mAP@50:95 25,6708%, thấp hơn Raw 42,5961% và 25,8215%. Kết quả cho thấy tăng chi tiết theo biên theo cấu hình hiện tại chưa cải thiện detector tổng thể. Không nên dùng độ sắc nét cảm nhận để thay cho thước đo phát hiện, cũng không dùng phép so chéo nhóm A/B để chọn một phiên bản tốt nhất cho mọi điều kiện.

5.2. Lộ trình kiểm chứng tiếp theo

Bảng 53. Các bước tiếp theo và tiêu chí hoàn tất

Ưu tiên	Công việc	Điều kiện đánh giá
1	Hoàn tất B-v2/B-v3 và khóa bảng cùng recipe	Marker, checkpoint hash, metric development; không đổi ngưỡng theo kết quả
2	Lặp nhiều seed cho Raw và ứng viên	Cùng ngân sách; báo cáo trung bình/độ lệch; không chỉ chọn seed tốt nhất
3	Paired bootstrap theo nhóm nguồn	Cùng ID/GT và prediction; giữ cấu trúc nhóm; ghi rõ uncertainty có điều kiện
4	Phân tích lỗi theo lớp, cỡ hộp, ánh sáng	Cùng ngưỡng vận hành; TP/FP/FN và đủ hỗ trợ; mô tả nhóm thiếu GT
5	Đo chi phí end-to-end	Đọc/giải mã + policy + JPEG nếu có + detector + hậu xử lý; cùng máy

Ưu tiên	Công việc	Điều kiện đánh giá
6	Chốt phương pháp trước final holdout	Không tune bằng final; chỉ mở khi có quyết định đánh giá cuối

Một thiết kế 2×2 hữu ích tách train-domain khỏi input-domain: mô hình train Raw/đọc Raw; train Raw/đọc processed; train processed/đọc Raw; train processed/đọc processed. So sánh hai ô dùng cùng checkpoint sẽ cô lập thay đổi input tốt hơn. Hiện các nhóm full50 chủ yếu cung cấp hai ô đường chéo; không giả định đã có đầy đủ bốn ô chỉ vì issue lịch sử có nghiên cứu detector cố định.

Nếu tiếp tục với edge detection, có thể đặt giả thuyết mới về ngưỡng Canny thích nghi hoặc biên đa tỉ lệ. Mỗi hướng phải có cấu hình cố định, budget và đối chứng; chưa có chứng cứ rằng nó sẽ cải thiện mAP. Học bộ tăng cường hoặc thêm nhánh đặc trưng biên vào mạng sẽ thay cả kiến trúc/huấn luyện, cần được đánh giá như nghiên cứu mới với đối chứng tương ứng. Tập exploration đã dùng không được gọi lại là tập xác nhận độc lập.

TÀI LIỆU THAM KHẢO

[1] F. Yu và cộng sự, “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” CVPR, 2020, tr. 2636–2645.
https://openaccess.thecvf.com/content_CVPR_2020/papers/Yu_BDD100K_A_Diverse_Driving_Dataset_for_Heterogeneous_Multitask_Learning_CVPR_2020_paper.pdf

[2] BDD100K, tài liệu/toolkit chính thức, mục Data Download và 100K Images. Truy cập 12/11/2026.
https://github.com/bdd100k/bdd100k/blob/master/doc/source/download.rst

[3] Ultralytics, “YOLO26” và tài liệu end-to-end detection. Truy cập 12/11/2026; thực nghiệm cố định package 8.4.198, không mặc định đồng nhất mọi thay đổi tài liệu mới.
https://docs.ultralytics.com/models/yolo26/

[4] OpenCV, “Changing the contrast and brightness of an image!”, mục Gamma correction. Truy cập 12/11/2026.
https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html

[5] OpenCV, “Histograms — Histogram Equalization”, mục CLAHE. Truy cập 12/11/2026.
https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

[6] J. Canny, “A Computational Approach to Edge Detection,” IEEE TPAMI, 8(6), tr. 679–698, 1986. DOI: 10.1109/TPAMI.1986.4767851.
https://www.andrew.cmu.edu/course/42-731/handouts/canny1986.pdf

[7] OpenCV, “Canny Edge Detection.” Truy cập 12/11/2026.
https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

[8] COCO Consortium, tài liệu và mã đánh giá detection. Truy cập 12/11/2026. Định nghĩa ngưỡng được tham chiếu; báo cáo dùng implementation metric native Ultralytics.
https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py

[9] Dự án vehicle-detection, notebook/YAML v0–v4, src/vehicle_detection/preprocess.py và issue v4 cục bộ. Mã tải từ repository; v4 và thực nghiệm đồng bộ là bổ sung cục bộ, không giả định đã được xuất bản lên GitHub.
https://github.com/truong51972/vehicle-detection

[10] Chứng cứ thực nghiệm cục bộ: configs/splits.csv.gz; runs/v4-2026-11-10; runs/v5_raw-2026-11-11; notebook output v0–v3 ngày 09/11/2026 và các alias nhóm B. Snapshot/hashes kèm báo cáo.

[11] Ultralytics, Object Detection with Ultralytics YOLO. Nguồn giới thiệu đầu ra detection và cách dùng model; truy cập 12/11/2026.
https://docs.ultralytics.com/tasks/detect/

[12] Ultralytics, Model Training with Ultralytics YOLO. Tài liệu train/fine-tune tham khảo; recipe thực nghiệm lấy từ YAML và args.yaml cục bộ, không từ mặc định web. Truy cập 12/11/2026.
https://docs.ultralytics.com/modes/train/

[13] Chứng cứ model cục bộ, Ultralytics8.4.198: yolo26.yaml, head.py, tasks.py, loss.py và validator.py; src/vehicle_detection/training.py; checkpoint COCO và B-Raw best.pt. Metadata/hashes trong model_evidence.json kèm báo cáo; chỉ đọc trên CPU, không tạo metric detection mới.

[14] Chứng cứ phương pháp cục bộ: data.py, preprocess.py, experiment_data.py, training.py, evaluation.py, evaluation_stats.py và YAML v0–v4; mã native matching/AP/fitness của Ultralytics8.4.198. Hash và hàm đã đối chiếu trong methodology_evidence.json; không chạy thêm train/inference khi soạn báo cáo.

PHỤ LỤC

A. Bảng đầy đủ Precision, Recall, F1 và AP theo lớp

Các bảng sau chép từ output DataFrame trong notebook 04_eval của nhóm A. Đơn vị toàn bộ cột metric là %. F1 là giá trị native đã lưu, không tính lại bằng P/R đã làm tròn. AP@75 được bổ sung để phân biệt kết quả ở ngưỡng IoU0,75 với trung bình AP@50:95.

Bảng 54. Nhóm A — v0: metric native đầy đủ theo lớp (%)

Class	P	R	F1	AP50	AP75	AP50:95
car	70,475	66,676	68,523	71,804	42,195	42,751
truck	54,982	45,532	49,813	46,380	35,650	32,314
bus	55,810	40,729	47,092	43,406	36,404	32,364
motorcycle	57,446	15,836	24,828	20,586	6,068	9,268
bicycle	51,671	24,825	33,537	27,998	8,796	12,268

Bảng 55. Nhóm A — v1: metric native đầy đủ theo lớp (%)

Class	P	R	F1	AP50	AP75	AP50:95
car	69,134	67,308	68,209	71,744	41,999	42,544
truck	53,888	44,543	48,772	45,712	34,948	31,850
bus	51,540	43,190	46,997	43,041	36,093	31,984
motorcycle	50,148	18,683	27,224	20,673	5,441	8,951
bicycle	54,244	23,130	32,431	27,418	8,166	11,858

Bảng 56. Nhóm A — v2: metric native đầy đủ theo lớp (%)

Class	P	R	F1	AP50	AP75	AP50:95
car	69,671	67,122	68,373	71,835	42,245	42,738
truck	55,220	45,214	49,718	45,657	34,770	31,753
bus	59,475	38,840	46,992	42,926	35,442	31,691
motorcycle	51,090	18,505	27,170	20,880	6,138	9,142
bicycle	54,032	25,152	34,325	28,510	8,632	12,037

Bảng 57. Nhóm A — v3: metric native đầy đủ theo lớp (%)

Class	P	R	F1	AP50	AP75	AP50:95
car	70,714	66,282	68,426	71,616	42,167	42,649
truck	53,879	44,543	48,768	45,935	34,985	31,750
bus	53,671	42,135	47,209	43,199	36,432	32,448
motorcycle	48,207	19,395	27,661	21,847	5,993	9,817
bicycle	58,056	22,456	32,386	28,077	8,161	11,978

B. Truy vết notebook và tọa độ output

Mỗi tọa độ dưới đây đếm cell và output từ 0 theo JSON .ipynb, không phải số In[] hiển thị của kernel. Đây là cách tìm đúng bảng kể cả khi notebook chưa mở hoặc chưa thực thi lại. Việc trích chỉ đọc output, không gọi model và không ghi lại notebook nguồn.

Bảng 58. Chỉ dẫn đọc kết quả trong từng notebook v0–v3

File	Cell / output (từ 0)	Chứng cứ
02_train_prepare.ipynb	cell4/output0	Run_date, đường dẫn và S20
02_train_prepare.ipynb	cell6/output0	Ảnh, tỷ lệ applied, brightness theo split
03_train.ipynb	cell2/output0	Training args đã hiển thị
03_train.ipynb	cell4/output image/png	Đường cong gốc; stream khi còn lưu
03_train.ipynb	cell6/output0	50 epoch, best calibration và epoch
04_eval.ipynb	cell2/output0	Evaluation settings
04_eval.ipynb	cell4/output1	Overall precision/recall/mAP50/mAP50_95
04_eval.ipynb	cell4/output2	Bảng từng lớp
04_eval.ipynb	cell6/output0	Cohort dark/bright và apply/skip nếu có
04_eval.ipynb	cell6/output1	Run path, speed_ms, final holdout sealed

Đường dẫn các file nhóm A là notebooks/experiments/vX/tên_file.ipynb. notebook_evidence.json đính kèm chứa hash SHA256, toàn bộ text output, bảng metric đã parse, nguồn hình và nhóm lịch sử. Hash chứng minh danh tính file được đọc; không tự chứng minh phép chạy gốc là đúng hoặc checkpoint gốc còn nguyên.

B.1. Hash nguồn v0

notebooks\experiments\v0\02_train_prepare.ipynb

```
SHA256: b3127289d2ea7af0e1f6d5101d3453b3fd4d74fc02309bc49efa37e3bbf7091a
```

notebooks\experiments\v0\03_train.ipynb

```
SHA256: 503bf619a869d2ff787fad4bc451bba995942fca290b20a45a38873d0df36ad2
```

notebooks\experiments\v0\04_eval.ipynb

```
SHA256: 9494da31ed89b3ca237066afc07d160d6ee6ed0efaef06ff7c07aee9057fa5b0
```

B.2. Hash nguồn v1

notebooks\experiments\v1\02_train_prepare.ipynb

```
SHA256: 04810be4a6ad5f8775915458c4ac9babd6fe93519fe081faa864e78d4a8e992d
```

notebooks\experiments\v1\03_train.ipynb

```
SHA256: d515630dee907de061e2c165b9b6946b8e7147764811b68f78a2f78059420e3a
```

notebooks\experiments\v1\04_eval.ipynb

```
SHA256: 37b3aaf40023217bb589dfd7d2050690bba11ad20494ade07c473baebb6c57e7
```

B.3. Hash nguồn v2

notebooks\experiments\v2\02_train_prepare.ipynb

```
SHA256: 1767bb231589b8c2376e92bd51c76c0bc51275730defee29920239c8d7c82245
```

notebooks\experiments\v2\03_train.ipynb

```
SHA256: 5c175d31d4541280e3239663f5097812686b23e8bff03ab8becdc800e9d80ccb
```

notebooks\experiments\v2\04_eval.ipynb

```
SHA256: 59865bfd5f4b8e11422134e72721be75c4f23a1ee0686d21e573549a54108a31
```

B.4. Hash nguồn v3

notebooks\experiments\v3\02_train_prepare.ipynb

SHA256: b716036d2bd9ad149e62970d462278ca12f4ca9ff5d9dab0f521594640bcb8e0

notebooks\experiments\v3\03_train.ipynb

SHA256: 7063f5f362174581d68bc210872d709725f841f4027a7b73e8495272f6d40d66

notebooks\experiments\v3\04_eval.ipynb

SHA256: 9b680b96b38f1b3b69d805619aaf0ee0b0e44a09a3f787d33f54ca0890f8fd81

C. Truy vết kết quả mới và dữ liệu cố định

evidence_snapshot.json ghi thời điểm chốt và hash kết quả nhóm B. Các alias v5_raw/v5_v1/v5_v2/v5_v3 là tên thư mục lượt chạy trong suite mới, vẫn tương ứng policy v0/v1/v2/v3; báo cáo không đổi tên phương pháp dựa theo alias.

v0: runs/v5_raw-2026-11-11; Hoàn tất trong nhóm B.

metrics SHA256: 4be2f68105f4f4d26e7bf6d09d2848bf3274baee07be658b52195fea088f781c

best.pt SHA256: 125d18b74d6fe85955b1bc09f8a1e603ce364f31899a69a3a26d47219706ed73

v1: runs/v5_v1-2026-11-11; Hoàn tất trong nhóm B.

metrics SHA256: 570c07c6c0b5fc3961ac9a83cf7885e8c3765e97f8d00f73e1e2a023abecd78b

best.pt SHA256: 18224e10e06d3a683b3bc3a03ac4b7e201031527bdfe6c15506424a5a91ac997

v2: runs/v5_v2-2026-11-11; Chưa hoàn tất trong nhóm B.

v3: runs/v5_v3-2026-11-11; Chưa hoàn tất trong nhóm B.

v4: runs/v4-2026-11-10; Hoàn tất trong nhóm B.

metrics SHA256: d1bb428835cb19612429b278a12a18ec6b6d2f74d84ba791c6587872c250007b

best.pt SHA256: 74a1338bf2645328509f2e16baf90b4c41ba6df7ae4c70e7e69f33d3e1049dfb

splits.csv.gz SHA256: b7a6aba1d1f72e1b1e3eceda8e8427d41f44408067fad63ee68479137714cb96

COCO yolo26n.pt SHA256: 9b09cc8bf347f0fc8a5f7657480587f25db09b34bf33b0652110fb03a8ad4fef

Mã chính sách tham chiếu: src/vehicle_detection/preprocess.py; cấu hình: notebooks/experiments/vX/vX.yaml với X=0…4; lifecycle: notebooks/experiments/v0 đến v4; issue v4 là tài liệu trong project. Snapshot không tự cập nhật theo tiến trình nền. Nếu metric của B-v2/B-v3 xuất hiện sau thời điểm chốt, đó là dữ liệu bổ sung cho bản tiếp theo, không phải lỗi thiếu kết quả nhóm A.

D. Cấu hình cần đối chiếu khi tái lập

Bảng 59. Checklist cấu hình và lý do phải giữ cố định

Nhóm	Tham số cần lưu	Tác động nếu thay đổi
Dữ liệu	ID, role, nhóm, nhãn năm lớp, JPEG95	Khác mẫu số/ảnh đầu vào, có thể đổi AP
Toán tử	Tất cả gate, scale512, clip/gain, kernel	Cùng tên thuật toán vẫn có thể khác pixel
Khởi tạo	COCO checkpoint, class remap, hash	Khác trọng số khởi tạo không còn đối chứng phù hợp
Train	Epoch50, freeze0, SGD, lr, warmup, augmentation	Khác learning dynamics và khả năng thích nghi
Batch/runtime	batch, nbs, worker, Python/Torch/OpenCV/CUDA	Có thể đổi thực thi và tái lập số học
Chọn model	Calibration policy, fitness, best epoch	Khác quy tắc chọn có thể thay checkpoint
Eval	imgsz640, batch, FP32, conf.001, IoU.7, maxdet300	Có thể đổi tập detection và tốc độ native

Bắt đầu bằng kiểm tra prerequisite của notebook 01→02→03→04 và dùng RUN_DATE riêng để không đè kết quả. Không thay YAML canonical batch32 rồi gọi số mới là bản lặp y hệt nhóm A. Đối với nhóm B dùng recipe alias của suite. Các thư mục ảnh export có thể đã được dọn sau đánh giá để tiết kiệm dung lượng; việc còn metric và checkpoint không đồng nghĩa ảnh processed còn sẵn, cần re-export đúng manifest trước phép suy luận chéo.

E. Nguồn lịch sử và cách diễn giải

H-v0: research\issue45-v2-pilot\docs\issue45-s20\v0_metrics.json

SHA256: 32067b0c3e7d0e4272839e85d40718bea68a4566af2b1f9b8f88c8ccb9090138

H-v1: research\issue45-v2-pilot\docs\issue45-s20\v1_metrics.json

SHA256: 889346972c912087503c792394ad8f70e4c80602c6abfdc9d9927e1641bd01d4

H-v2: research\issue45-v2-pilot\docs\issue45-s20\v2_metrics.json

SHA256: 96c87a47fb0183d846c0ae5f3e58ed92cafbc6d4c818dd5cd14f98c848ca80fc

Các tài liệu mô tả sử dụng thêm: nhánh research/issue47-v3-960, docs/ISSUE45_S20_EXECUTION.md và docs/ISSUE47_V3_960_RESULTS.md; nhánh feat/issue-5-data-protocol, docs/ISSUE5_RESULTS.md. Các bản sao cùng nội dung ở nhiều nhánh không được đếm thành nhiều lần chạy độc lập. Những bảng lịch sử phục vụ giải thích tiến trình nghiên cứu; không gộp chúng vào trung bình kết quả của full fine-tune.

F. Hạn chế và hoàn thiện hồ sơ nộp

Báo cáo sử dụng trực tiếp template Word FPT được cung cấp; giữ bìa, hệ thống chương, tóm tắt ngắn, mục lục, danh mục hình/bảng và tài liệu tham khảo. Tên nhóm, họ tên thành viên và MSSV chưa được cung cấp nên để ô điền rõ ràng; tên giảng viên giữ đúng template. Các trường TOC/SEQ được cập nhật khi xuất PDF. Người thực hiện cần hoàn thiện thông tin định danh trước khi nộp.

Kết luận thực nghiệm chỉ dùng số đo có nguồn. Không có độ chính xác test cuối, không có khoảng tin cậy mới của full50 và không có đo end-to-end. Để cập nhật báo cáo, bổ sung nguồn metric có provenance, tính delta trong đúng nhóm, cập nhật bảng/hình cùng lúc rồi kiểm tra lại kết luận; không chỉ thay một con số trong phần tóm tắt.

G. Hồ sơ model và cách đọc mã train

model_evidence.json lưu metadata đọc trên CPU, hash checkpoint, YAML kiến trúc và nguồn loss cục bộ ở thời điểm mở rộng báo cáo. Đây là chứng cứ bổ sung về model; số liệu thực nghiệm vẫn dùng snapshot đã chốt ngày12/11/2026 lúc15:39:25 UTC+7. Không chạy lại training hay inference để viết phần này.

Checkpoint khởi tạo: weights/yolo26n.pt

```
SHA256: 9b09cc8bf347f0fc8a5f7657480587f25db09b34bf33b0652110fb03a8ad4fef
```

Checkpoint B-Raw đã fine-tune: runs/v5_raw-2026-11-11/train/weights/best.pt

```
SHA256: 125d18b74d6fe85955b1bc09f8a1e603ce364f31899a69a3a26d47219706ed73
```

Lời gọi cốt lõi của project nằm trong train_experiment của src/vehicle_detection/training.py: nạp YOLO(checkpoint), verify_coco_names(model.names), rồi model.train(data=..., project=..., name="train", exist_ok=True, **settings). settings lấy từ YAML của version và seed; wrapper còn kiểm tra hash, dữ liệu chuẩn bị, trạng thái hoàn tất và việc resume. Vì thế ví dụ ngắn model.train mặc định trên mạng không đủ để tái lập toàn bộ kết quả của đồ án.

Bảng 60. Các file cần đọc để hiểu detector và thực nghiệm

Nguồn	Nội dung cần đọc
weights/yolo26n.pt	Checkpoint COCO khởi tạo, taxonomy80
src/vehicle_detection/training.py	Kiểm tra model nguồn, setting và lifecycle train
src/vehicle_detection/evaluation.py	Thiết lập validation và xuất metric/cohort
notebooks/experiments/vX/vX.yaml	Cấu hình policy, seed, train và eval của version
ultralytics/cfg/models/26/yolo26.yaml	Cấu trúc module, scale n, reg_max, các đầu ra P3/P4/P5
ultralytics/nn/modules/head.py	Head phân lớp/hồi quy, nhánh one-to-one và giải mã
ultralytics/utils/loss.py	E2ELoss, v8DetectionLoss và BboxLoss trong runtime
ultralytics/engine/validator.py	Chọn chế độ end-to-end theo args.nms

Đường dẫn package có thể thay đổi khi cài môi trường mới; hash file cụ thể trong model_evidence.json giúp nhận diện đúng nguồn đã kiểm tra. Tài liệu web được dùng để giới thiệu họ model, còn mô tả setting và trạng thái thực nghiệm ưu tiên cấu hình, mã và checkpoint của project. Benchmark COCO công khai không được dùng thay cho kết quả BDD năm lớp của đồ án.

H. Nguồn chi tiết cho Chương3

methodology_evidence.json chứa danh sách file và hash SHA256 đọc khi mở rộng phương pháp, kèm các hàm native matching/AP/fitness đã đối chiếu. Các sơ đồ và ví dụ số trong chương 3 nhằm

giải thích quy trình, không tạo thêm kết quả detection. Mã project, notebook và tiến trình train nền không bị sửa trong lần viết này.

Bảng 61. Ánh xạ nội dung methodology sang hàm thực thi

Nội dung	**Hàm / nguồn chính**
Chia cố định và chống lẫn vai trò	shared_splits, source_rows, populations trong experiment_data.py
Hình học / nhãn / ảnh nền	annotation_profile, remap_boxes trong data.py; verify_label
Quality và toán tử v0–v4	quality, apply_recipe, local_shadow, local_contrast, canny_detail
Chuẩn bị và toàn vẹn export	_export, prepare_data, verify_data
Huấn luyện / resume / marker	train_experiment, validate_training_args, completed_training
Eval và cohort	evaluate_experiment, cohort_metrics trong evaluation.py
Lưu matching và tổng hợp AP	MatchingStats, class_ap trong evaluation_stats.py
Matching/AP/fitness native	BaseValidator.match_predictions; ap_per_class; compute_ap; Metric.fitness

Một giới hạn truy vết đã được nêu rõ: metadata export hiện chưa chứa edge_density; operator_sha256 là trường ghi nhận chứ chưa được verify_data tự đối chiếu với mã hiện tại. Đây là mô tả trung thực implementation tại thời điểm đọc, không phải cam kết rằng các kiểm tra chưa có đã được triển khai. Hash ảnh/nhãn và source manifest vẫn giúp xác định chứng cứ đã sử dụng.